[DATE]

BOOLEAN LOGIC CIRCUIT SIMULATOR

AQA COMPUTER SCIENCE NEA

KOZELL YON-BROOKS
5727
51319

Contents

Contents 2
Analysis 4
The Problem 4
End Users 4
Overview of Logic Gates 4
Features Needed for GCSE Students 5
Analysis of existing systems 7
Acceptable Limitations 8
Hardware and software constraints for Highdown 8
Futureproofing for mobile development 8
Platform, Programming Language, and Modules 9
Kivy Framework 9
Overview 9
Kivy Language 9
Objectives 9
General 9
Specific 10
Design 12
Overview 12
Hierarchy Chart 12
Input, Output, Process, Storage 13
Object Oriented Design 13
Logic Class Diagram 14
User Interface 15
Main Interface 15
Gate Canvas 16
Toolbar 16
Menubar for adding components 16
Menubar for changing tool 16
Truth Table Popup 17
Components 18
Nodes 19
Data Structures 20
Logic Components 20
Truth Table 21

Algorithms 22
Truth Table Generator: 22
File Structure 25
Technical Solution 26
Truth Table 26
Logic Board 28
Logic Components 29
GUI (Python) 37
GUI (kvlang) 54
Testing 62
Overview 62
Code Testing Table 62
Video Testing Table 63
Code Testing Results 67
Code Testing Code 67
Video Testing Link 74
Evaluation 75
Overview 75
Objectives Not Met 75
Objectives Partially Met 75
Improvements That Can Be Made 75
User Feedback 76

Analysis

The Problem

GCSE Computer science is where a student's introduction to logic gates may happen, this is a part of the curriculum but also helps to further a student's logical thinking which can help in other areas of the subject, such as programming. A way to help me better understand logic gates were logic gate simulators, these simulators allowed me to create simple circuits using basic AND, OR, XOR, and NOT gates virtually. A simulator would allow the students to create circuits that give automatically give outputs without needing to be traced. However, the current simulators are too complex for what is needed for GCSE computer science and lack the teaching tools that my proposed system will include.

End Users

The primary users of the logic gate simulators will be GCSE students studying computer science. However, as the computer science teachers will most likely introduce the students to this program and use it as part of their lessons, they will be considered as end-users as well.

Overview of Logic Gates

Logic gates are a model of computation that take one or two inputs and returns a single output based on the gate's logical operation / Boolean function, they are the fundamentals of logical circuits and physical logic gates made of diodes and transistors are what allow computers to work. Logic gates can be combined to produce a certain output based on the inputs of the circuit. An AND gate (Figure 1) for example will take two inputs and return a True output if both inputs are True, and a False output otherwise. Inputs and outputs can be True or False as they are Boolean, this is usually represented as a 1 and 0 for True and False, respectively. Certain gates such as the Not gate will only need one input. There are other parts to the logic circuits besides the gates; switches, constant inputs, and clocks can provide initial inputs. Output can be handled by a simple 'bulb' that is on/lit for True and off for False. A more complex output such as a 4-bit digit would produce an integer output based on a binary sequence from 4 Boolean inputs.

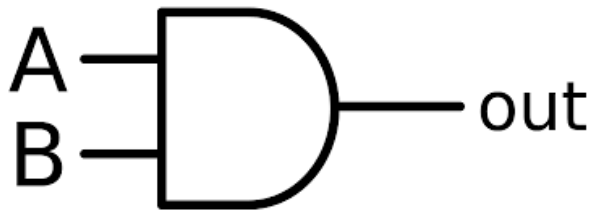


Figure 1: AND Logic Gate

Features Needed for GCSE Students

The truth table below (Figure 2) displays all the possible inputs and outputs in tabular form for the Logic operations AND, OR, XOR, and NOT. These are the only gates that are needed for the AQA course as per the specification (Figure 3).

Input		Output			
A	B	AND	OR	XOR	NOT (Input only A)
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Figure 2: Example of Truth Table

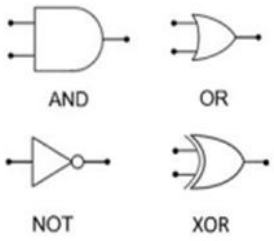
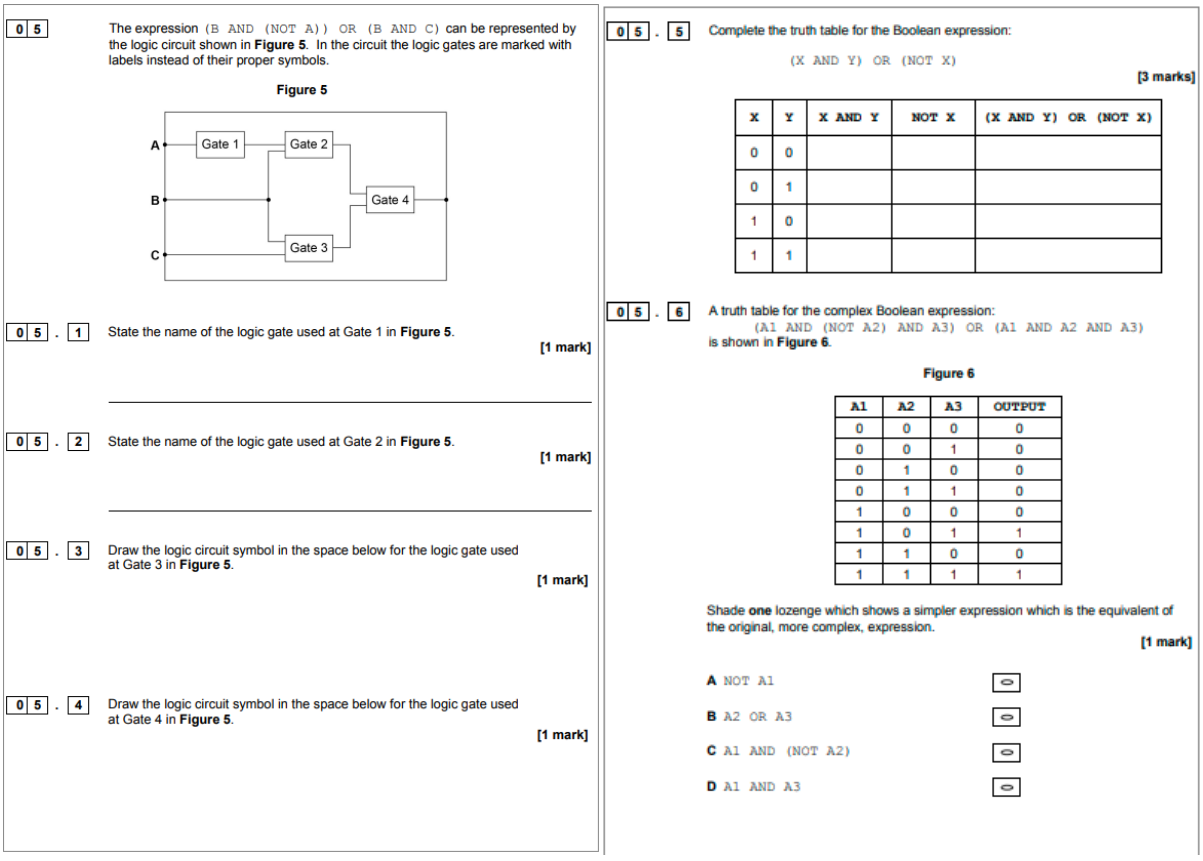
Content	Additional information
Create, modify and interpret simple logic circuit diagrams. Students will only need to use NOT, AND, OR and XOR gates within logic circuits. Students will be expected to understand and use the following logic circuit symbols:  AND OR NOT XOR	Students should be able to construct simple logic circuit diagrams which contain up to three inputs.
Create and interpret simple Boolean expressions made up of NOT, AND, OR and XOR operations.	Students will be expected to understand and use the following Boolean expression operators: . to represent the AND gate + to represent the OR gate ⊕ to represent the XOR gate Overbar to represent the NOT gate For example the expression (A AND B) OR (NOT C) would be represented as: $(A \,.\, B) + \bar{C}$
Create the Boolean expression for a simple logic circuit. Create a logic circuit from a simple Boolean expression.	

Figure 3: AQA GCSE Specification for Boolean Algebra 3.4.2

As these are the only gates needed for the AQA course, they will be the only gates included, simplifying the program for the students. My solution would be more focused on teaching, specifically what is needed for the AQA GCSE specification. I had a short conversation with Mr Flynn about what features would make my program relevant to GCSE students. These included:

- Abstracting logic gates that are not needed.
- Including a checklist to determine if the user has used each of the gates at least once.
- A truth table generator to display the results of a circuit as a truth table.
- Converting a written Boolean expression into a truth table.
- Displaying the Boolean expression for the created circuit.
- Allowing individuals to save and load circuits to and from their computer's local storage.
- Comparing a user-made circuit to a given Boolean expression to check if they have created the circuit correctly

These are all the features I believe are needed for the student to understand and solve GCSE level questions such as the one shown in Figure 4 below.



0 5 The expression (B AND (NOT A)) OR (B AND C) can be represented by the logic circuit shown in **Figure 5**. In the circuit the logic gates are marked with labels instead of their proper symbols.

Figure 5

0 5 . 1 State the name of the logic gate used at Gate 1 in **Figure 5**. **[1 mark]**

0 5 . 2 State the name of the logic gate used at Gate 2 in **Figure 5**. **[1 mark]**

0 5 . 3 Draw the logic circuit symbol in the space below for the logic gate used at Gate 3 in **Figure 5**. **[1 mark]**

0 5 . 4 Draw the logic circuit symbol in the space below for the logic gate used at Gate 4 in **Figure 5**. **[1 mark]**

0 5 . 5 Complete the truth table for the Boolean expression:

(X AND Y) OR (NOT X)

[3 marks]

X	Y	X AND Y	NOT X	(X AND Y) OR (NOT X)
0	0			
0	1			
1	0			
1	1			

0 5 . 6 A truth table for the complex Boolean expression:

(A1 AND (NOT A2) AND A3) OR (A1 AND A2 AND A3)

is shown in **Figure 6**.

Figure 6

A1	A2	A3	OUTPUT
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Shade **one** lozenge which shows a simpler expression which is the equivalent of the original, more complex, expression. **[1 mark]**

A NOT A1

B A2 OR A3

C A1 AND (NOT A2)

D A1 AND A3

Figure 4: Logic Gate question from AQA GCSE Computer Science June 2019 (8020/1)

Analysis of existing systems

Figure 5, Figure 6, and Figure 7 below are examples of existing logic gate simulators.

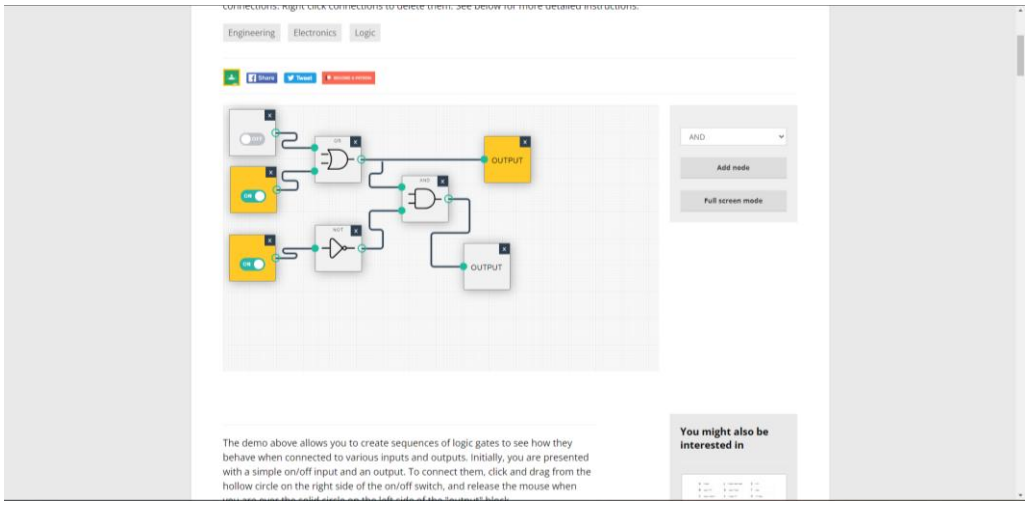


Figure 5: https://academo.org/demos/logic-gate-simulator/

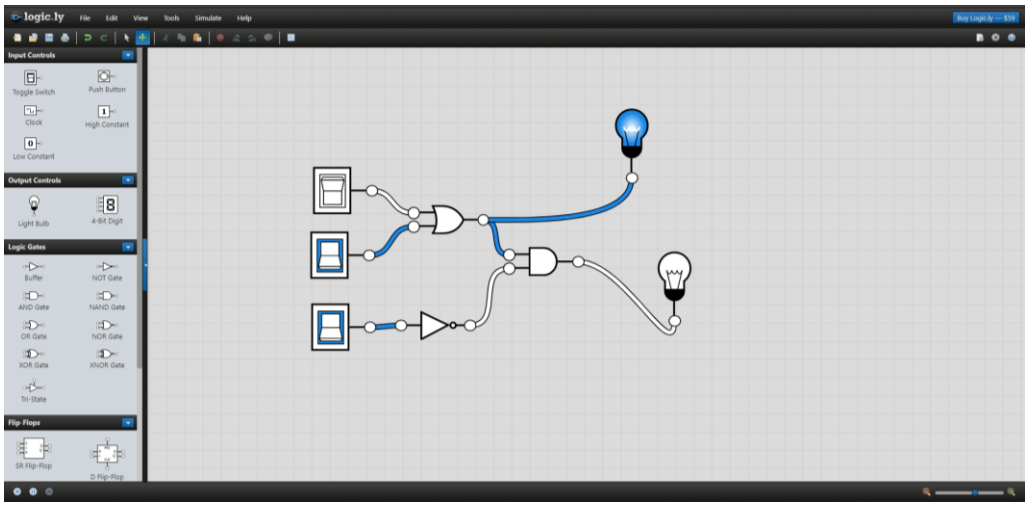


Figure 6: https://logic.ly/demo/

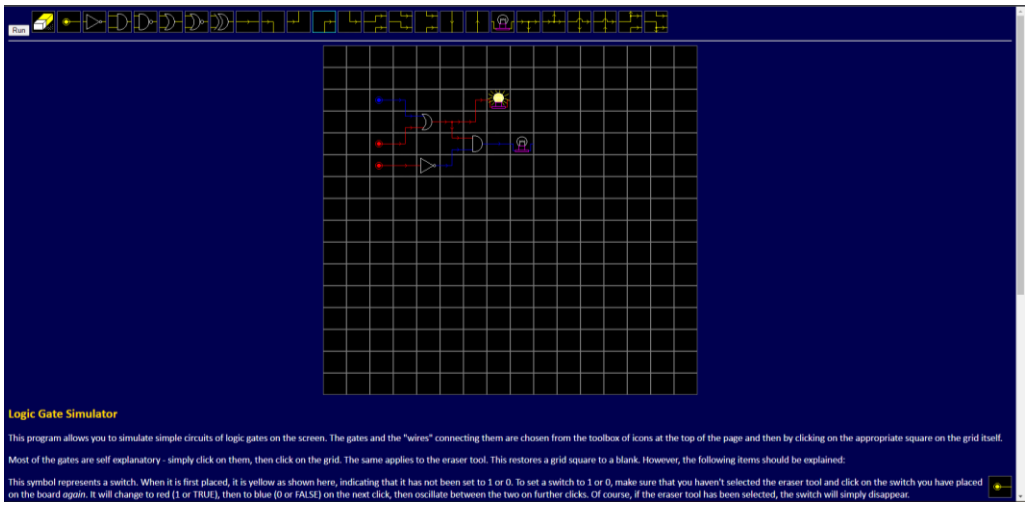


Figure 7: http://www.richardbowles.co.uk/resources/digital/tools/sim/sim.html

	Advantages	Disadvantages
Figure 5	• Lines between gates clearly show the connections of gates.	• Gates are behind a drop-down menu, making them harder to find. • No function to generate a truth table from the circuit created.
Figure 6	• Lines connecting gates are coloured to show the value of the gates they are coming out of.	• Gate selection is too complex for a GCSE student.
Figure 7	• Lines connecting gates are coloured to show the value of the gates they are coming out of. • Gates are clearly shown in the menu bar at top of the page.	• No dragging of gates around the canvas; makes repositioning difficult. • States of components do not dynamically update; the run button must be pressed to update them when a change is made. • Wires are placed manually; this is tedious and makes repositioning components difficult.

Acceptable Limitations

Hardware and software constraints for Highdown

The program must run on school computers and therefore not be resource-intensive due to their performance. The school uses Windows 10 as its operating system, so the program currently does not need to run on any other OS.

Futureproofing for mobile development

Because I would like the program to be run on mobile devices such as phones, the interface must be interacted with by only one button – the left-click on the mouse. This means each input must be a variation of a mouse click, release, drag, double-click, etc. This should allow easy transposing into touch inputs when altering the program for mobile development.

Platform, Programming Language, and Modules

The program would run as either a web application or a windows desktop application. These choices are suitable as Highdown uses windows as the operating system on its computers, and all its computers have a browser and constant access to the internet.

A desktop application is a type of software that is directly installed onto the hard drive of the computer. It can be launched whenever, independent of other applications, i.e., it does not need a web browser to run within, like a web app. They also work regardless of internet connection to access it (unless the program itself requires it).

A Web application is a type of software application that is used through the internet via a web browser. The files are stored on a remote server, the backend processing is done remotely, and the application is only accessible via an internet connection and browser.

As a desktop application will provide all the features needed for the program, I have chosen it over the web app as it does not need a server to be stored/run on nor require a constant internet connection to access.

The programming language I use will somewhat depend on the platform that the program will be run on. For a web app, the programming languages I would use would be Python, JavaScript, and HTML; I would also need to be familiar with CSS. For a windows application, many languages would be suitable, including python. As python is a language that I am already familiar with, it is my top choice for programming language, because the program will be a desktop application only python is needed. Python version 3.9 is the minimum version of python that I will use.

For the GUI, python has an array of frameworks to help build user interfaces. These include PyQt5, tkinter, Kivy, wxPython, and PySimpleGUI. Kivy is currently my choice as it has many notable features and allows for the possibility of deploying the program on multiple platforms (Windows, macOS, iOS, Android). https://kivy.org/#home

Kivy Framework

Overview

Kivy is a free and open-source Python framework for developing mobile apps and other multitouch application software with a natural user interface. It is distributed under the terms of the MIT License, and can run on Android, iOS, Linux, macOS, and Windows. - Wikipedia.

Kivy Language

Kivy has its own declarative language, KvLang, which dedicated to describing the interface of the program. This means the python file does not need all the code for static interface items, this makes the python code shorter and easier to read.

Objectives

General

The general objective is to create a program that will allow a user to create logical gate circuits using draggable components that will evaluate and return an output based on the circuit. The program will also show a truth table of an expression either taken from a component or input manually. The program should be responsive, easy to use, compatible with the school computers, and have the gates that are on the AQA GCSE computer science specification.

Specific

1. The program must create and evaluate logic gate circuits.
    1.1. Can have gates, switches, and/or outputs (the three referred to as components).
        1.1.1. Each component will have a state of either true or false.
        1.1.2. The gates will be AND, OR, NOT, and XOR (from GCSE spec).
        1.1.3. The components can connect to each other.
            1.1.3.1. The components will have input and output nodes that point to their connected gates.
        1.1.4. The components can return their state as an output.
        1.1.5. The gates can calculate their state based on the gate's Boolean operator and its inputs through an 'evaluate' method.
        1.1.6. The switches can be flipped, changing their state from false to true or vice-versa.
        1.1.7. Each component stores the expression for the circuit up to where it is in the circuit.
            1.1.7.1. The expression will update when components are connected and disconnected
            1.1.7.2. The components operator will be correctly added to the expression.
    1.2. There will be a board class containing the components.
        1.2.1. The board will store the created components in an array.
        1.2.2. The board will tell the gates to connect and disconnect to and from each other.
        1.2.3. The board will remove the components from its array and disconnect everything it connected to.
2. There will be a truth table generator.
    2.1. It will produce a truth table for a given expression.
        2.1.1. It can use the expression of an output component.
        2.1.2. It can use an input Boolean expression string.
    2.2. It will produce a list of input combinations based on the number of inputs.
    2.3. It will substitute each combination of inputs into the expression to get an output.
    2.4. It will list the outputs alongside the input combinations in a table.
3. There will be a graphical user interface.
    3.1. It will allow for the dynamic placing of the components onto a canvas.
        3.1.1. It will use mouse inputs to drag the components and move them about the canvas.
        3.1.2. Connection lines will be added between connected components.
            3.1.2.1. They will show the state of the component they're exiting through their colour.
            3.1.2.2. They will not be straight lines; they will have a 90-degree bend at its horizontal centre to keep the canvas tidy and help with finding where the lines lead to.
    3.2. It will have a component toolbar that will have buttons that can add components to the canvas.
        3.2.1. The gates will have an indicator showing whether they have been used in the current session.
    3.3. It will have a tool toolbar that will determine what will happen when the components are interacted with depending on the tool selected.
        3.3.1. Connect tool: the program will tell the board to connect the selected components.
        3.3.2. Disconnect tool: the program will disconnect the selected components from each other.
        3.3.3. Move tool: the selected component can be moved about the canvas.
        3.3.4. Delete tool: the selected components will be deleted.

3.3.5. Clear tool: All components will be deleted.
3.3.6. Truth Table tool: the program will show a truth table for a given expression.
3.3.6.1. It will have a text input box that the user can enter their own expression into
3.3.6.2. It will take the expression from a selected component when opened.
3.4. It will have a menu bar with options:
3.4.1. Save, which saves the current circuit to a JSON file.
3.4.2. Load, which loads a circuit from a JSON file.
3.4.3. Help, which shows an information page about how to use the program.
3.4.4. Quit, which exits the program.
3.4.4.1. When clicked, a popup will appear asking if you are sure you want to exit.
3.5. The user interface will be resizable with scrollable menus to allow access but keep the text legible.

Design

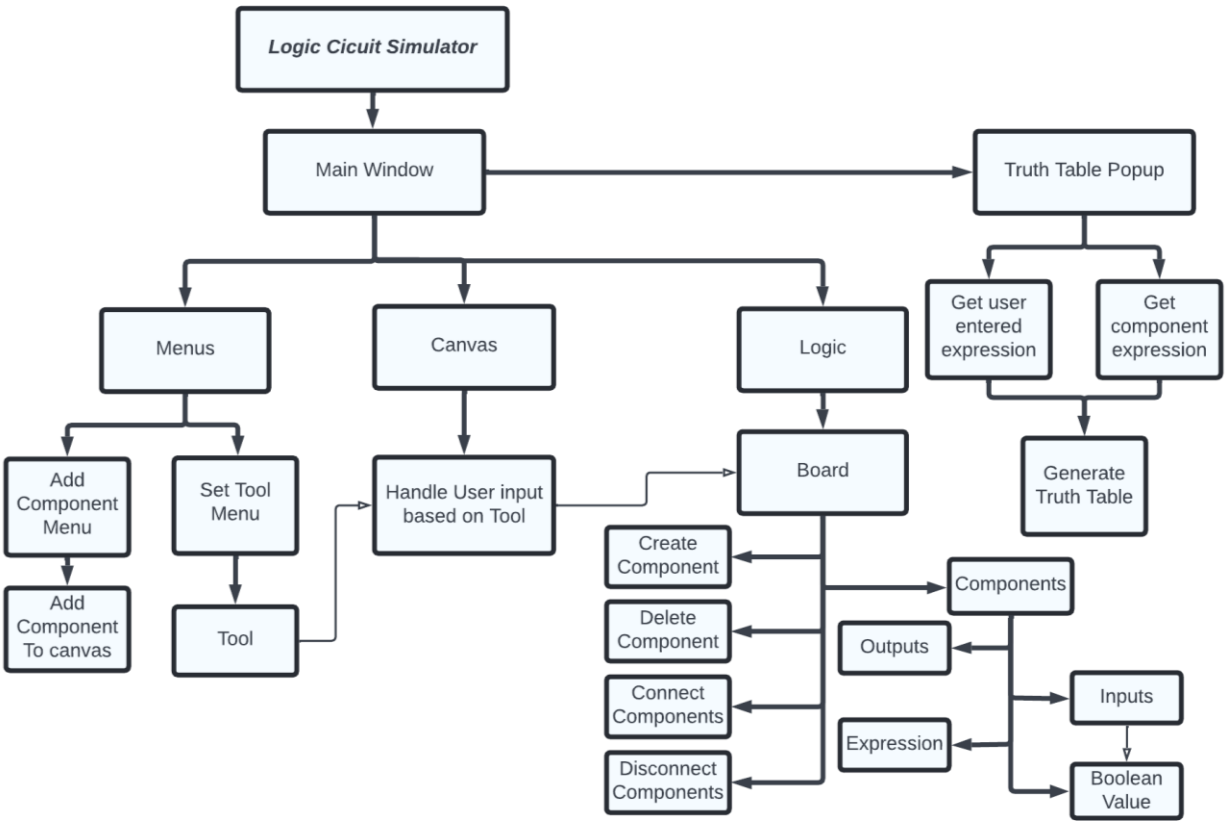
Overview

This design section will show how the system will look and operate, and demonstrate how the objectives will be implemented.

I have structured the project so that the logic of the circuits are independent of the user interface, this allows the logic code to be used in different project and to be tested without the whole program.

The main window will house everything the user can interact with. The gate canvas will have a board object, this will control the logic component objects. The board will tell them to connect and disconnect, but the components will store who they are connected to themselves. A logic component will be created when a visual component is created and stored in the component array of the board. When the canvas interacts with the visual component, the canvas will ask for its logic component reference and pass that to the board. The board will use that reference to get the logic component from its component array for use.

Hierarchy Chart



Input, Output, Process, Storage

As Kivy can be used to develop for multiple operating systems, and I would like the program to be easily converted into an Android or iOS app, the user should be able to interact with the program with a single mouse button being pressed or released, and its movement around the screen, as this simulates what can be done on a touch screen.

Input	Output
Mouse Inputs: Left Click Down, Mouse Move, Left Click Up. These inputs will be used for moving the components around a canvas and connecting or disconnecting them through dragging or clicking. (3.1.1) Button Presses: Buttons around the user interface are what will control most of the program functionality, such as adding gates, selecting tools, and opening the truth table menu. (3.2, 3.3, 3.4) Boolean expression input box: The truth table menu will display a truth table created for a user-entered Boolean expression; this will be a string entered into a text box which the truth table generator will then use as the input expression. (2.1.2)	Graphical Indicators: Most of the outputs will be done through the GUI, changing colours of components and lines based on their states, buttons showing as depressed/coloured to indicate when they are pressed, and borders around components when they are selected or being interacted with. (3.1) Truth Table: A multiline string representation of a truth table produced by the truth table generator function will be displayed. The string will only be printed onto the screen. (2.4)
Process	**Storage**
Truth Table Generator: Will process a Boolean expression into a truth table. (2) Boolean Evaluation: Each gate will use the values of their inputs with their respective Boolean operators to produce their output value. (1.1.5)	Saving and loading files to a centralised system is a stretch goal for this project and may not be implemented. The primary goal is a simulation tool to support students investigating simple logic gate circuits they will develop as part of their studies.

Object Oriented Design

The program will be written with an object-oriented design pattern. The program will be split into modules for the UI and Logic. Within these modules classes are used to define the components of the program such as the individual logic components and the widgets of the kivy framework. The program will make use of object-oriented concepts such as classes, abstraction, inheritance, and polymorphism.

Logic Class Diagram

Figure 8 below shows the class diagram for the logic side of the program. The 4 gates (and, or, xor, not) will inherit from a parent class called Gate, the subclasses will contain the same attributes as the parent class but will overwrite some to fit the gates purpose, e.g. And_Gate will overwrite the Gate classes currently empty attribute *type* with the string “and”. The gate subclasses will also overwrite the evaluate method so that it uses the correct boolean operator. As the not gate is different to the other 3 gates because of its single input rather than 2, it will overwrite other methods to account for only having 1 input.

There are also classes for switch and output objects, these have methods similar to the gate class, but with slight changes as they have only one node and no boolean evaluation method. Switch also has a flip method which will *flip* its state.

The switch, output and all gate classes all have a relationship with the Board class through composition. The board class will have 2 attributes, board_id; an integer, and gates; an array of all components that currently exist. It will also have methods to add, remove, connect, and disconnect gates, a method to clear the board which will remove all components.

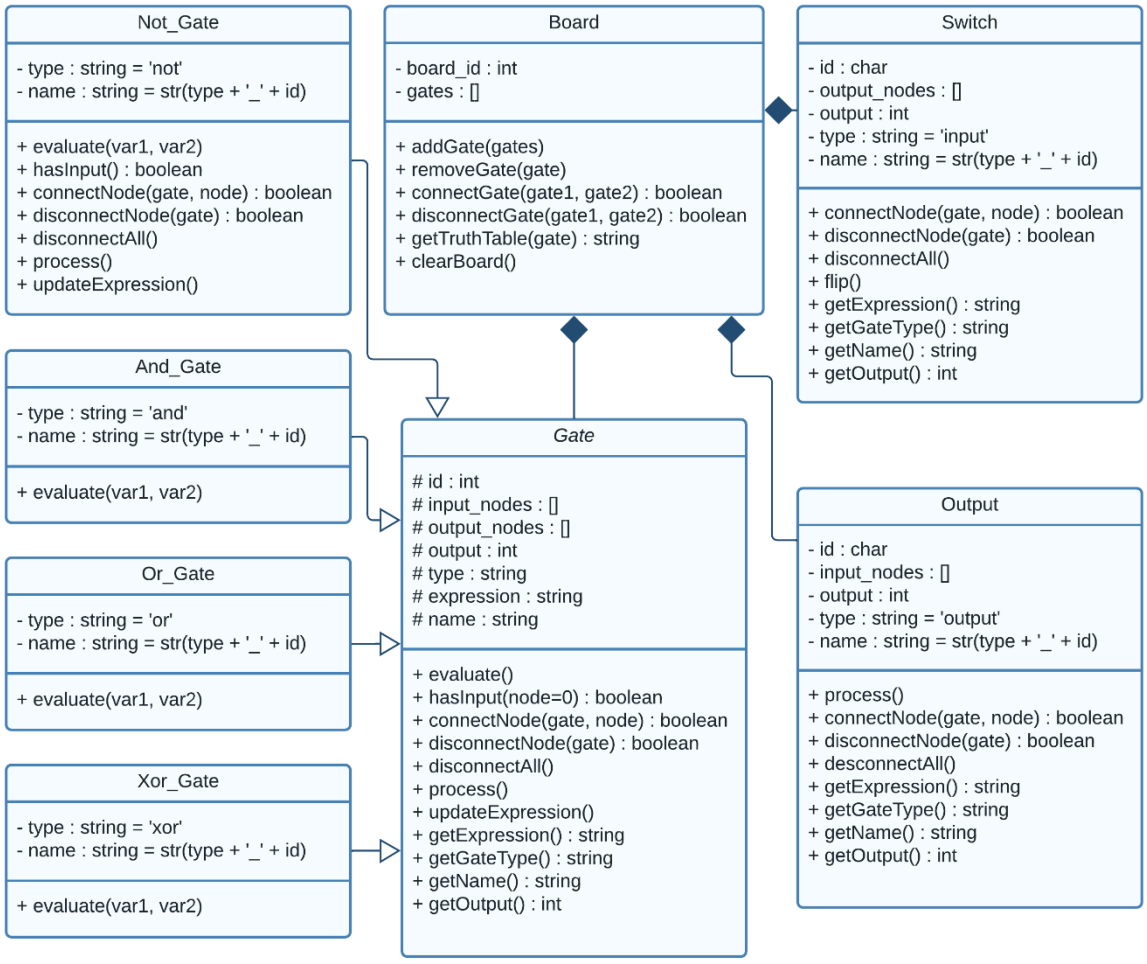


Figure 8: Class diagram for circuit components

User Interface

Main Interface

When the program is initially opened, the first thing seen will be the full interactable program, allowing the user to start creating their circuits right away. The user interface should be simple and intuitive to use, no features should be hidden away or unclear on what their use is. Figure 9 shows a draft of how the main interface will look and the sections it will be split into.

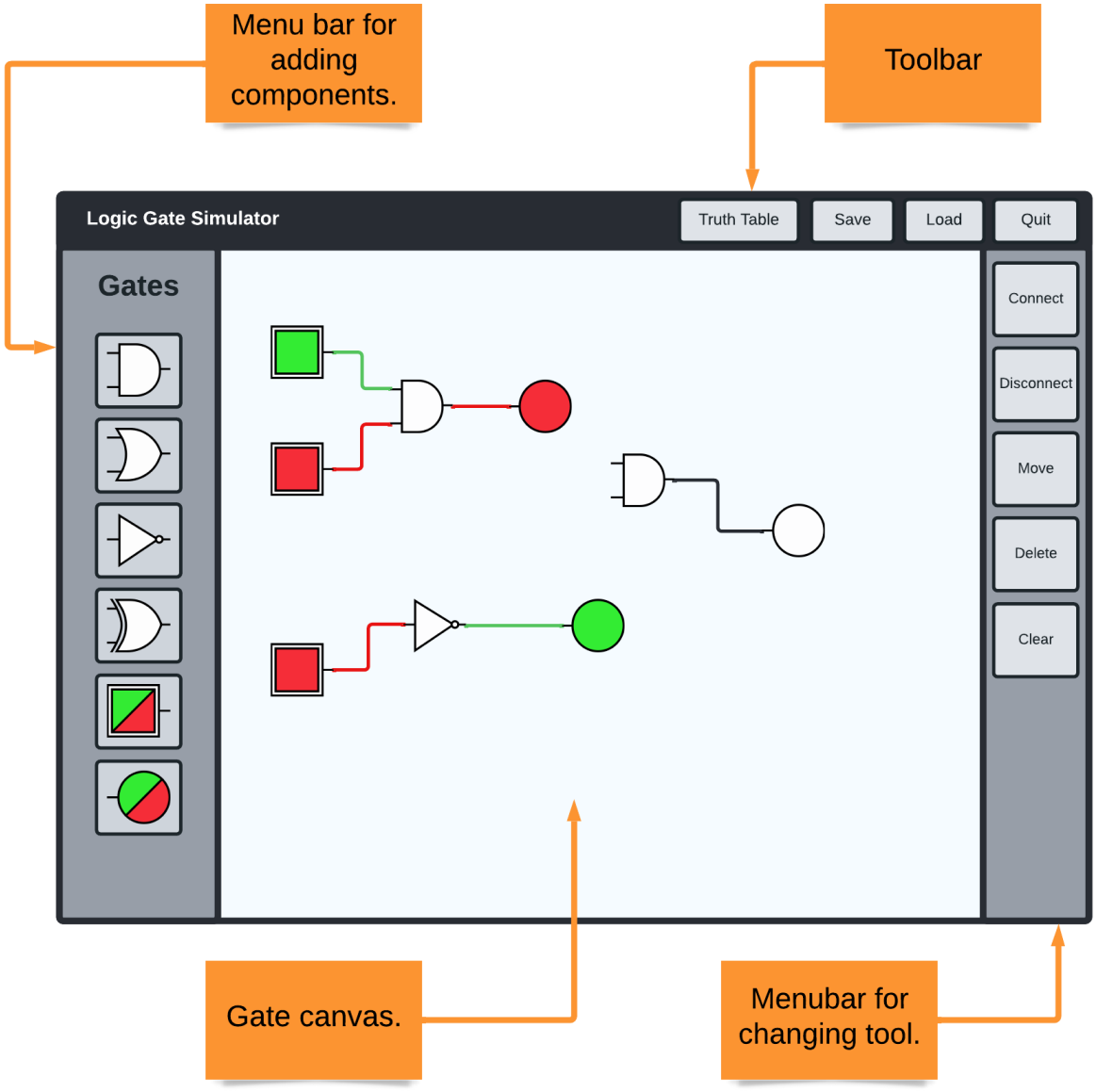


Figure 9: Main Graphical User Interface Design

Gate Canvas

This is the canvas that will display Gates and Connection Lines. It will allow the gates two be placed anywhere within the canvas and the connection lines will dynamically update their positions to match where the components they are connected to are positioned.

Toolbar

This will house the buttons to save, load, and open the truth table popup interface. It also has a quit button to quit the program, this will also bring up a popup asking for confirmation that the user wants to quit and that anything unsaved will be lost.

Menubar for adding components

This sidebar will contain the components that the user can add to the canvas, it will have buttons that when pressed adds the component to the canvas. Each button will also have an indicator that shows whether the component has been used in the current session. There will also be labels alongside the components to show what they are but this isn't shown in Figure 9.

Menubar for changing tool

This is a sidebar with 5 buttons, the three top buttons are toggles that the user can choose between (one and only one of the three will always be on). The bottom two buttons, delete and clear, are not toggles and will immediately have an effect. Below is an explanation of each tool.

Tool	What it does.
Connect	When toggled, the gate canvas will show the nodes of each component, the user may then drag from a node of one component to the node of another to connect them. This will create a connection line on the canvas linking the two components. The order of the connecting nodes (OUT → IN or IN → OUT) should not matter, the program will handle connecting them correctly. Connecting an out-node to an out-node or in-node to an in-node will not work. The components should not allow multiple connections to their in-nodes but should allow multiple connections from their out-nodes.
Disconnect	When toggled, the gate canvas will display the nodes of each component, the user may then click a node of a component and it will disconnect that component from the component that its node is connected to. When the components are disconnected, the connection lines connected to the clicked node will be removed. This means if the user clicks an out-node with multiple connections out of it, it will remove all connections.
Move	When toggled, the nodes are hidden, the user may click a component to select it, multiple components can be selected, click-and-drag a component to move it around the canvas, and double click the switch to flip its state.
Delete	When pressed, this will remove any selected components from the canvas. It will only work if the move tool is toggled. If not, it will toggle the move tool but will not remove the components.
Clear	When pressed, all components on the canvas will be deleted. Will toggle move tool when gate canvas is cleared.

Truth Table Popup

Truth Table Popup: This is where the user can generate truth tables. The expression input box will allow the user to input a Boolean expression, when the user presses the enter key or clicks the generate button a truth table for that expression will display in the truth table area. If a component is selected when the truth table button is pressed in the toolbar of the main interface, the expression box will automatically be populated with the expression of that component. Only one component must be selected, or it will default to empty on opening. There is also a cancel button to close the popup.

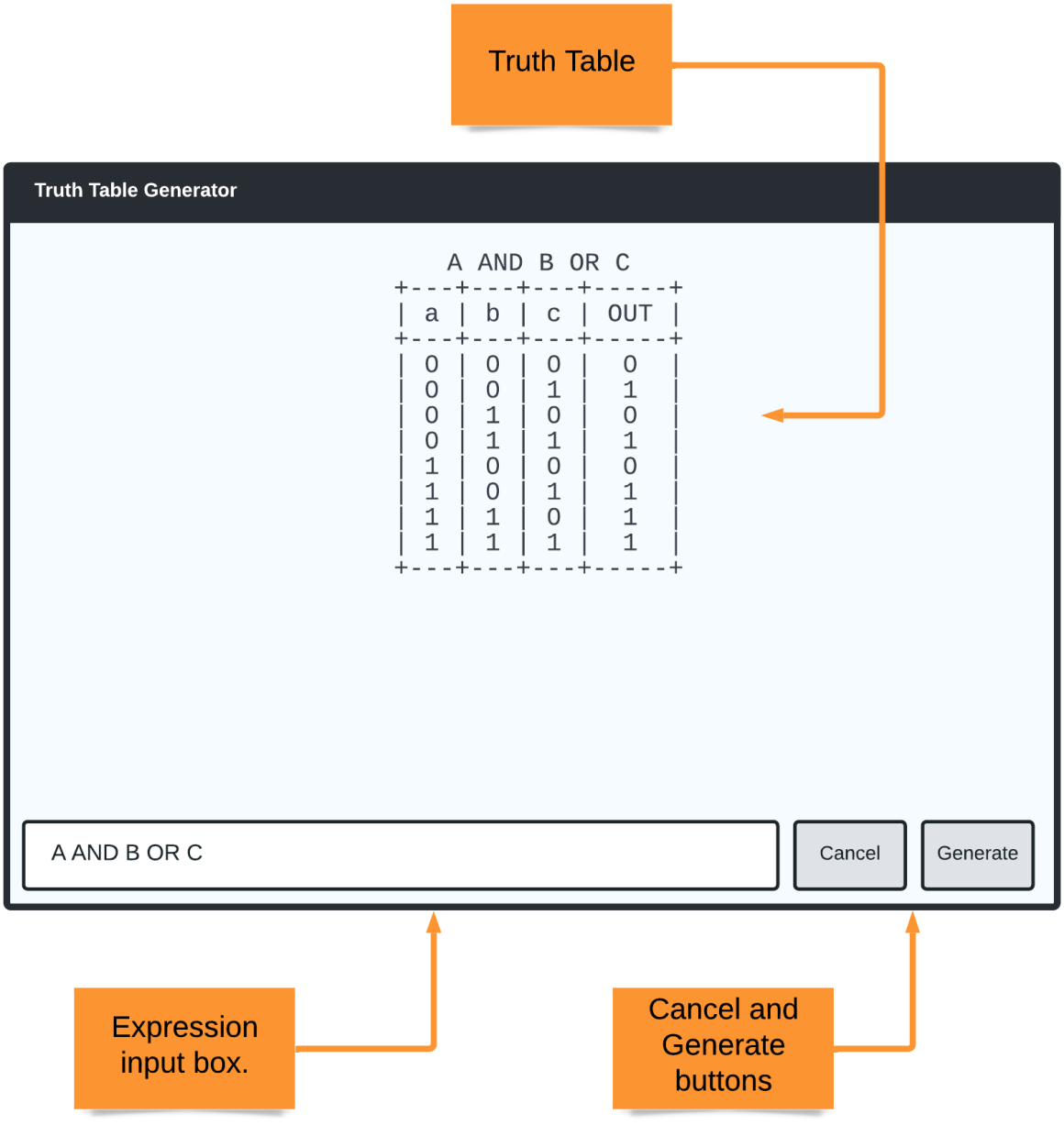
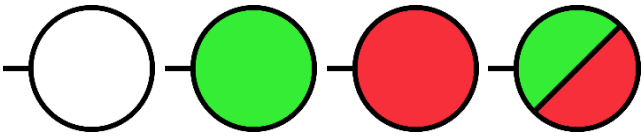
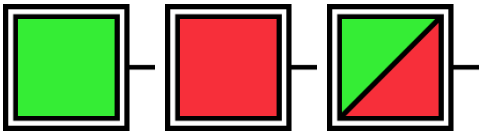
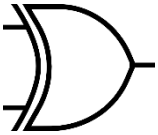
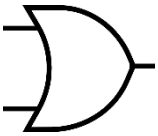
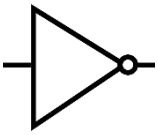
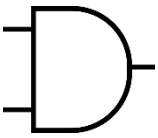


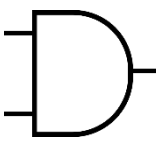
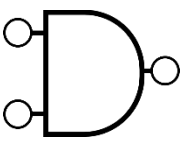
Figure 10: Truth Table popup UI

Components

These are all the icons for the components. The menu icons for switch and output will only appear in the menu bar for adding components. The gate components will only have one icon as they do not need to show their state. Switch and output will have Green and Red versions to show their states, output will also have an extra white icon for when its state is None (when the circuit is incomplete). They are all PNGs as they benefit from lossless compression and have transparency.

Component	Icons
Output: (None, On, Off, Menu)	
Switch: (On, Off, Menu)	
XOR Gate	
OR Gate	
NOT Gate	
AND Gate	

Nodes: Component inputs and outputs are split into nodes, an AND gate for example will have 3 nodes, input_node_1, input_node_2, and out_node, whereas a SWITCH or OUTPUT will have only an out_node or in_node_1, respectively. The nodes are how the user will connect and disconnect components. The nodes will only appear when either the connect or disconnect tool is selected.

And Gate	And Gate with visible Nodes
	

Nodes

Component nodes will be visually linked by connection lines; these lines will be drawn onto the canvas and will move with the components so that they always start on the out_node of one component and end on the correct in_node of the other component. Figure 11 below shows how the line's colour will show the state of the gate it's exiting, green for true, red for false, and black/white for an incomplete or invalid circuit. The lines will also shape themselves to always be made up of only horizontal or vertical parts. This will help with following the lines and organising the canvas.

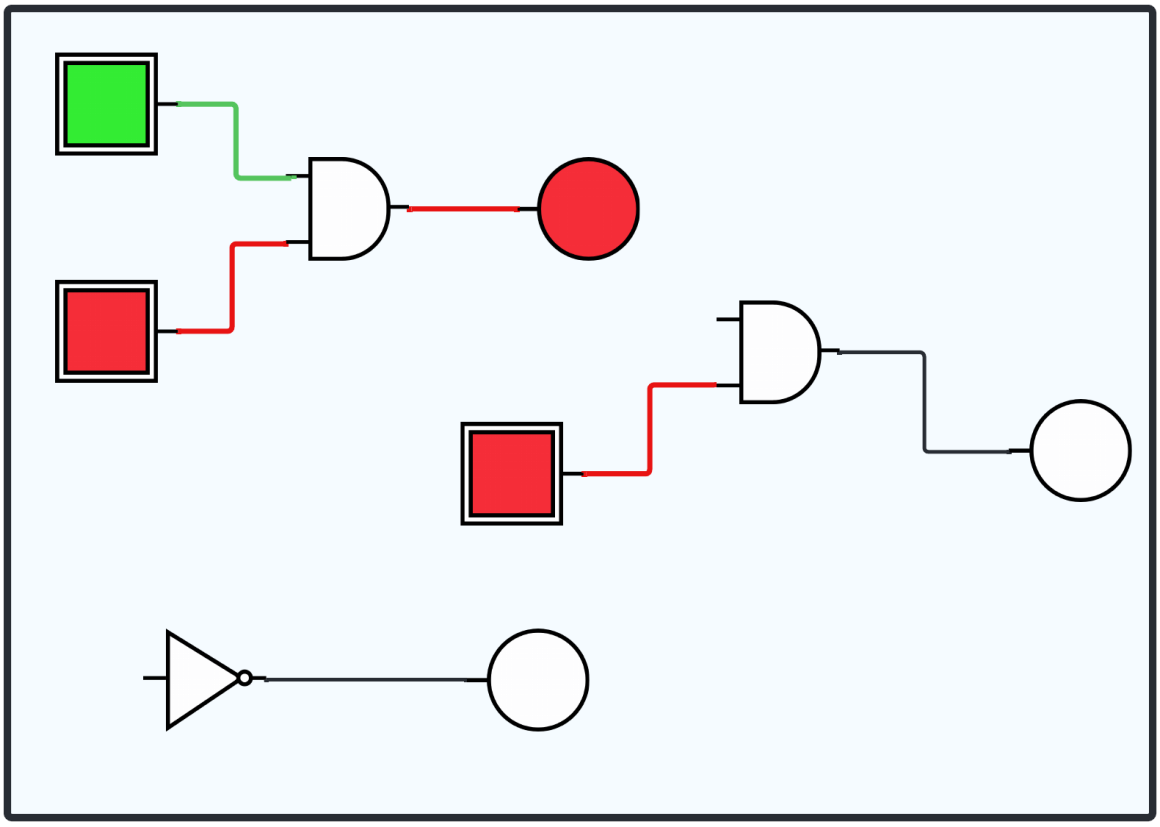


Figure 11: 3 Logic Circuits

Data Structures

Logic Components

The logic components in a circuit will be treated as a binary tree. The outputs or last component in a circuit would act as the root node, each node would represent a component and would have either 2 children (AND, OR, XOR), 1 child (NOT) or no children (SWITCH). This conforms to the binary tree structure.

Because each component can have multiple output connections and therefore each node can have multiple parents, the overall structure does not fully match a binary tree. However when the logic is evaluated the logic will work from the root component and will not need access to any output components of the components.

A preorder traversal recursive method will check if each component has all its input connections, if not it will return its current boolean output. If the component does have both input connections it will call a preorder traversal method on the left child/1st input. When the left child has returned some data, it will store it and then repeat for the right child/2nd input if there is one. Once all of its children have returned data, the component will use it in an evaluate method to get a boolean value and return that.

Below is a pseudocode representation of a node that has two inputs.

```
CLASS Node2:
      instance.data = [None, None]
      instance.left = None
      instance.right = None

      FUNCTION set_data(data):
            instance.data = data

      FUNCTION set_left(left_node):
            instance.left = left_node

      FUNCTION set_right(right_node):
            instance.right = right_node

      FUNCTION preorder_traverse():
            IF (instance has no left or right) THEN:
                  RETURN instance.data
            ELSE
                  instance.data[0] ← instance.left.preorder_traverse()
                  instance.data[1] ← instance.right.preorder_traverse()
                  RETURN instance.data
```

Truth Table

The truth table will be stored using a dictionary. The dictionary will have keys that are strings, which is valid as strings are hashable, and values which are arrays of integers. The final key value pair will store the outputs of the expression.

```
Expression = “A or (B and C)”
Example_Dictionary: dict[key, list[int]] = {
      “A”:[0,0,0,0,1,1,1,1],
      “B”:[0,0,1,1,0,0,1,1],
      “C”:[0,1,0,1,0,1,0,1],
      “OUT”:[0,0,0,1,1,1,1,1]
}
```

This translates into a truth table where each dictionary entry is a column of the table:

A or (B and C)			
A	**B**	**C**	**OUT**
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Algorithms

Truth Table Generator:

```
FUNCTION generateTruthTable(expression):
	input_expression ← expression
	expression ← expression.lower()
	expression ← expression.replace("not", "2+~")
	expression ← expression.replace("and", "&")
	expression ← expression.replace("xor", "^")
	expression ← expression.replace("or", "|")
	operators ← ["2+~", "&", "^", "|", "(", ")"]
	variables ← []
	temp_exp ← expression.replace("(", "")
	temp_exp ← temp_exp.replace(")", "")
	temp_exp ← temp_exp.split(" ")
	FOR(i IN temp_exp):
		IF (i NOT IN operators) AND (i NOT IN variables):
		variables.append(i)
	num_variables ← len(variables)
	temp ← []
	FOR(i ← 0 TO num_variables):
		temp.append(0)
	FOR(i ← 0 TO num_variables):
		temp.append(1)
	p ← get_permutations(temp, num_variables)
	variable_permutations ← []
	FOR(i IN p):
		IF i NOT IN variable_permutations:
			variable_permutations.append(i)
	dictionary = DICTIONARY()
	FOR i,j IN ENUMERATE(variables):
		FOR ii IN variable_permutaions:
			dictionary[j] = i[ii]
	final_dictionary ← getTruthTableOutputs(expression, dictionary,
LEN(variable_permutations))
```

Explanation:

The function is given a Boolean expression as a string. The first 7 lines will format the string into an expression that can be used by the getTruthTableOutputs function.

An array called variables is created and populated with each variable in the expression, they are found by removing any brackets and splitting the string by its spaces into an array. Then each item is checked if it's in the list of operators and if not, added to variables.

The number of variables is found from the length of the variables array and a temporary array will be created with that amount of 0s followed by the same amount of 1s. This is for the get_permutations function to work correctly.

The get_permutations will return a list of the permutations of the temp array as tuples of size num_variables. E.g., for an expression with 2 variables:

```
get_permutations([0,0,1,1], 3) will return

[(0, 0), (0, 1), (0, 1), (0, 0), (0, 1), (0, 1), (1, 0), (1, 0), (1, 1), (1, 0), (1, 0), (1, 1)]
```

The duplicated tuples are then removed to make the variable_permuations array:

```
[(0, 0), (0, 1), (1, 0), (1, 1)]
```

This method was used as it produced a nicely ordered list of all possible value combinations used in a truth table.

A dictionary is then made with the variables as keys, and an array of their ordered values, found by combining the values with the same index in the variable_permutations array tuples.

E.g. with variables A and B:

```
{'A':[0,0,1,1], 'B':[0,1,0,1]}
```

The formatted expression, dictionary, and the number of tuples in the variable_permutations will be given to the getTruthTableOutputs function which will return the same dictionary but with an extra OUT key with its values.

A tuple of the final dictionary and the original expression will then be returned.

```
FUNCTION getTruthTableOutputs(expression, dictionary, num_permutations):
      truth_dictionary ← dictionary
      list_of_outputs ← []
      FOR (i ← 0 TO num_permutations):
            expression_with_variables ← expression
            FOR (variable in truth_dictionary):
                  value ← INT_TO_STRING(dictionary[variable][i]))
                  expression_with_variables ←
                  expression_with_variables.replace(variable, value)
            TRY:
                  output ← EVALUATE(expression_with_variables)
            EXCEPT SyntaxError as e:
                  return "Error"
            list_of_outputs.append(output)
      truth_dictionary['OUT'] ← list_of_outputs
      RETURN truth_dictionary
```

Explanation:

The function is given a formatted expression, a dictionary with a key-value pair of each variable and the ordered array of its values, and num_permutations which is the number of values the variables will have.

The function creates a copy of the given dictionary called truth_dictionary and an array called list_of_outputs. Within a for loop counting from i = 0 to the num_permutations, a copy of the expression called expression will be created, a for loop iterating through the truth_dictionary variables will then produce a value by converting the value at index i of the dictionary value with the variable as key to a string. All occurrences of that variable in the expression_with_variables string are replaced the current value the for loop has. This is done for all other variables so that the expression is now solveable. An evaluate function will treat the expression as code. If the expression cannot be correctly evaluated, most likely due to an invalid expression the function will return ‘ERROR’ and not continue. If the expression is evaluated, the output of it is added to a array called list_of_outputs. The process is repeated for all other values that the variable can be. The now completed list_of_outputs array is now added to the truth dictionary under the key ‘OUT’.

The updated truth_dictionary is returned.

File Structure

| CODE ¬
    | Logic ¬
        | TruthTable.py
        | Components.py
        | Board.py
    | Gate Icons ¬
        | (All component icon PNGs)
    | GUI.py
    | GUI.kv

This is how the file structure will be laid out for the program code. The GUI.py will import the code from each file in the Logic folder and access the icons of the component from the Gate Icons folder.

Technical Solution

Truth Table

```
from itertools import permutations
from os import system as sys
from typing import Union, Tuple

def get_tt_outputs(expression: str, dictionary: dict[str, list],
num_permutations: int) -> dict[str, list]:
    '''Adds OUT variable and values to incomplete truth table dictionary using
by replacing variables in given expression.'''
    truth_dictionary = dictionary
    list_of_outputs = []
    for i in range(num_permutations):
        expression_with_variables = expression
        for variable in truth_dictionary:
            expression_with_variables =
expression_with_variables.replace(variable, str(dictionary[variable][i]))
        try:
            output = eval(expression_with_variables)
        except SyntaxError as e:
            print(e, expression_with_variables)
            return "Error"
        list_of_outputs.append(output)
    truth_dictionary['OUT'] = list_of_outputs
    return truth_dictionary


def generate_truth_table(expression: str) -> Union[Tuple[dict[str, list],
str], str]:
    '''Returns a truth table in dictionary form'''
    input_expression = expression
    expression = expression.lower()
    expression = expression.replace("not ", "2+~ ")
    expression = expression.replace("not", "2+~ ")
    expression = expression.replace("and", "&")
    expression = expression.replace("xor", "^")
    expression = expression.replace("or", "|")

    operators = ["2+~", "&", "^", "|", "(", ")"]
    variables = []

    temp_exp = expression.replace("(", "").replace(")", "")
    for var in temp_exp.split(" "):
        if var not in operators and var not in variables:
            variables.append(var)
```

```
    num_variables = len(variables)

    temp = [0 for _ in range(num_variables)] + [1 for _ in
range(num_variables)]
    perms = permutations(temp, num_variables)
    variable_permutations = []
    for perm in perms:
        if perm not in variable_permutations:
            variable_permutations.append(perm)

    dictionary = dict()
    for i, key in enumerate(variables):
        dictionary[key] = [perm[i] for perm in variable_permutations]

    final_dictionary = get_tt_outputs(expression, dictionary,
len(variable_permutations))

    if final_dictionary == "Error":
        return "Invalid Input"
    else:
        return (final_dictionary, input_expression)


if __name__ == '__main__':
    '''For Testing'''
    sys('clear')
    exp = input('Here: ')
    print(generate_truth_table(exp))
    print()
```

Logic Board

```
from logic.gates import *
from logic.truth_table import *

class Board:
    '''Logic Board class.'''
    ID = 0
    def __init__(self) -> None:
        self.id = Board.ID
        Board.ID += 1
        self.gates = []

    def add_gate(self, gate: Gate) -> None:
        '''Adds given component to gate array.'''
        self.gates.append(gate)

    def remove_gate(self, gate: Gate) -> None:
        '''Removes given component from gate array.'''
        gate.disconnect_all()
        self.gates.remove(gate)

    def connect_gate(self, in_gate: Gate, out_gate: Gate, node: int) -> bool:
        '''Connects two given components together'''
        if node == 1:
            if in_gate.connect_node(out_gate, 1):
                return True
        if node == 2:
            if in_gate.get_gate_type() != "not" or in_gate.get_gate_type() !=
"output":
                if in_gate.connect_node(out_gate, 2):
                    return True
        return False

    def disconnect_gate(self, in_gate: Gate, out_gate: Gate) -> bool:
        '''Disconnects two given components from eachother'''
        return in_gate.disconnect_node(out_gate)

    def clear_board(self) -> None:
        '''Removes all components from gate array'''
        for gate in self.gates[:]:
            self.remove_gate(gate)
```

Logic Components

```
from typing import Union # For type hinting

class Gate:
    '''Logic Gate component class. Parent class for logic gates.'''
    ID = 0
    def __init__(self) -> None:
        self._input_nodes = [None, None]
        self._output_nodes = []
        self._output = None
        self._type = ''
        self._expression = None
        self.name = ''
        self.id = Gate.ID
        Gate.ID += 1

    def evaluate(self) -> None:
        '''Not implemented in parent class.'''
        pass

    def has_input(self, node=0) -> bool:
        '''Checks if nodes are connected to other components.'''
        if node == 0:
            return bool(self._input_nodes[0] and self._input_nodes[1])
        elif node == 1:
            return bool(self._input_nodes[0])
        elif node == 2:
            return bool(self._input_nodes[1])

    def connect_node(self, gate, node) -> bool:
        '''Connects given component to given node of self'''
        if gate in self._input_nodes:
            return False
        if node == 1:
            if not self.has_input(1):
                self._input_nodes[0] = gate
                gate.connect_node(self, -1)
                self.update_expression()
                return True
            else:
                return False
        elif node == 2:
            if not self.has_input(2):
                self._input_nodes[1] = gate
                gate.connect_node(self, -1)
                self.update_expression()
                return True
```

```
            else:
                return False
        elif node == -1:
            self._output_nodes.append(gate)
            return True

    def disconnect_node(self, gate) -> bool:
        '''Disconnects given component from self.'''
        if self._input_nodes[0] is gate:
            self._input_nodes[0].disconnect_node(self)
            self._input_nodes[0] = None
            self.update_expression()
            return True
        elif self._input_nodes[1] is gate:
            self._input_nodes[1].disconnect_node(self)
            self._input_nodes[1] = None
            self.update_expression()
            return True
        elif gate in self._output_nodes:
            self._output_nodes.remove(gate)
            return True
        else:
            return False

    def disconnect_all(self) -> None:
        '''Disconnects all inputs and outputs from self.'''
        if self._input_nodes[0] != None:
            self._input_nodes[0].disconnect_node(self)
            self._input_nodes[0] = None
            self.update_expression()
        if self._input_nodes[1] != None:
            self._input_nodes[1].disconnect_node(self)
            self._input_nodes[1] = None
            self.update_expression()
        for gate in self._output_nodes:
            gate.disconnect_node(self)

    def _process(self) -> None:
        '''Uses evaluate method to set output of self. If both input nodes
empty, or any input nodes output is None, output is set to None.'''
        if self.has_input():
            var1 = self._input_nodes[0].get_output()
            var2 = self._input_nodes[1].get_output()
            if var1 == None or var2 == None:
                self._output = None
            else:
                self._output = self.evaluate(var1, var2)
        else:
```

```
            self._output = None

    def update_expression(self) -> None:
        '''Updates the boolean expression of self. Used when connecting or
disconnecting components to/from self.'''
        if self.has_input():
            if self._input_nodes[0].get_gate_type() == 'switch':
                exp1 = self._input_nodes[0].get_expression()
            else:
                exp1 = f"({self._input_nodes[0].get_expression()})"
            if self._input_nodes[1].get_gate_type() == 'switch':
                exp2 = self._input_nodes[1].get_expression()
            else:
                exp2 = f"({self._input_nodes[1].get_expression()})"

            self._expression = f"{exp1} {self._type} {exp2}"
        else:
            self._expression = None

    def get_expression(self) -> str:
        '''Returns boolean expression of self.'''
        return self._expression

    def get_gate_type(self) -> str:
        '''Returns the type of component that self is.'''
        return self._type

    def get_name(self) -> str:
        '''Returns the name of self. Name is string: type_id.'''
        return self.name

    def get_output(self) -> Union[int, None]:
        '''Calls process method, then return output of self.'''
        self._process()
        return self._output


class And_Gate(Gate):
    '''Logic And_Gate component class, child of Gate class.'''
    def __init__(self) -> None:
        super().__init__()
        self._type = 'and'
        self.name = f"{self._type}_{str(self.id)}"

    def evaluate(self, var1, var2) -> int:
        '''Return the result of an and comparison bewtween two variables.'''
        return int(var1 and var2)
```

```
class Or_Gate(Gate):
    '''Logic Or_Gate component class, child of Gate class.'''
    def __init__(self) -> None:
        super().__init__()
        self._type = 'or'
        self.name = f"{self._type}_{str(self.id)}"

    def evaluate(self, var1, var2) -> int:
        '''Return the result of an or comparison bewtween two variables.'''
        return int(var1 or var2)


class Xor_Gate(Gate):
    '''Logic Xor_Gate component class, child of Gate class.'''
    def __init__(self) -> None:
        super().__init__()
        self._type = 'xor'
        self.name = f"{self._type}_{str(self.id)}"

    def evaluate(self, var1, var2) -> int:
        '''Return the result of an xor comparison bewtween two variables.'''
        return int((var1 and not var2) or (not var1 and var2))


class Not_Gate(Gate):
    '''Logic Not_Gate component class, child of Gate class.'''
    def __init__(self) -> None:
        super().__init__()
        self._type = 'not'
        self.name = f"{self._type}_{str(self.id)}"

    def evaluate(self, var1) -> int:
        '''Return the result of a not operation on a variable.'''
        return int((not var1))

    def has_input(self) -> bool:
        '''Checks if input node is connected to another component.'''
        return bool(self._input_nodes[0])

    def _process(self) -> None:
        '''Uses evaluate method to set output of self. If has no input node,
or input nodes output is None, output is set to None.'''
        if self.has_input():
            var1 = self._input_nodes[0].get_output()
            if var1 == None:
                self._output = None
            else:
                self._output = self.evaluate(var1)
        else:
```

```
            self._output = None

    def connect_node(self, gate, node) -> bool:
        '''Connects given component to given node of self.'''
        if gate in self._input_nodes:
            return False
        if node == 1:
            if not self.has_input():
                self._input_nodes[0] = gate
                gate.connect_node(self, -1)
                self.update_expression()
                return True
            else:
                return False
        elif node == -1:
            self._output_nodes.append(gate)
            return True

    def disconnect_node(self, gate) -> bool:
        '''Disconnects given component from self.'''
        if self._input_nodes[0] is gate:
            self._input_nodes[0].disconnect_node(self)
            self._input_nodes[0] = None
            self.update_expression()
            return True
        elif gate in self._output_nodes:
            self._output_nodes.remove(gate)
            return True
        else:
            return False

    def disconnect_all(self) -> None:
        '''Disconnects all inputs and outputs from self.'''
        if self._input_nodes[0] != None:
            self._input_nodes[0].disconnect_node(self)
            self._input_nodes[0] = None
            self.update_expression()
        for gate in self._output_nodes:
            gate.disconnect_node(self)

    def update_expression(self) -> None:
        '''Updates the boolean expression of self. Used when connecting or
disconnecting components to/from self.'''
        if self.has_input():
            self._expression =
str(f"{self._type}({self._input_nodes[0].get_expression()})")
        else:
            self._expression = None
```

```
class Switch:
    '''Logic Switch component class.'''
    ID = 0
    def __init__(self) -> None:
        self._output = 0
        self._output_nodes = []
        self._type = 'switch'
        self.id = chr(Switch.ID+97)
        Switch.ID += 1
        self.name = (f"{self._type}_{self.id}")

    def connect_node(self, gate, node) -> bool:
        '''Connects given component to given node of self.'''
        if gate not in self._output_nodes and node == -1:
            self._output_nodes.append(gate)
            return True
        return False

    def disconnect_node(self, gate) -> bool:
        '''Disconnects given component from self.'''
        if gate in self._output_nodes:
            self._output_nodes.remove(gate)
            return True
        return False

    def disconnect_all(self) -> None:
        '''Disconnects all inputs and outputs from self.'''
        for gate in self._output_nodes:
            gate.disconnect_node(self)

    def flip(self) -> None:
        '''Changes output of self from 0 to 1 or 1 to 0.'''
        self._output = int(not(self._output))

    def get_name(self) -> str:
        '''Returns the name of self. Name is string: type_id.'''
        return self.name

    def get_gate_type(self) -> str:
        '''Returns the type of self that self is.'''
        return self._type

    def get_expression(self) -> str:
        '''Return character id of self.'''
        return self.id

    def get_output(self) -> Union[int, None]:
```

```
        '''Returns output of self.'''
        return self._output


class Output:
    '''Logic Output component class.'''
    ID = 65
    def __init__(self) -> None:
        self._output = None
        self._input_nodes = [None]
        self._type = 'output'
        self.id = chr(Output.ID)
        Output.ID += 1
        self.name = (f"{self._type}_{self.id}")

    def _process(self) -> None:
        '''Sets output of self to output of input node. If input node is
empty, output is set to None.'''
        try:
            self._output = self._input_nodes[0].get_output()
        except AttributeError as e:
            print(e, len(self._input_nodes))
            self._output = None

    def connect_node(self, gate, node) -> bool:
        '''Connects given component to given node of self.'''
        if self._input_nodes[0] == None:
            self._input_nodes[0] = gate
            gate.connect_node(self, -1)
            return True
        else:
            return False

    def disconnect_node(self, gate) -> bool:
        '''Disconnects given component from self.'''
        if self._input_nodes[0] is gate:
            self._input_nodes[0] = None
            gate.disconnect_node(self)
            return True
        return False

    def disconnect_all(self) -> None:
        '''Disconnects all inputs and outputs from self.'''
        if self._input_nodes[0] == None:
            pass
        else:
            self._input_nodes[0].disconnect_node(self)
            self._input_nodes[0] = None
        self._process()
```

```
    def get_expression(self) -> str:
        '''Return boolean expression of input component.'''
        return self._input_nodes[0].get_expression()

    def get_name(self) -> str:
        '''Returns the name of self. Name is string: type_id.'''
        return self.name

    def get_gate_type(self) -> str:
        '''Returns the type of component that self is.'''
        return self._type

    def get_output(self) -> Union[int, None]:
        '''Calls process method, then return output of self.'''
        self._process()
        return self._output
```

GUI (Python)

```
import sys
import platform
import kivy
kivy.require('2.0.0')

from kivy.app import App
from kivy.clock import Clock
from kivy.config import Config
from kivy.core.window import Window
from kivy.graphics import Color, Line
from kivy.lang import Builder
from kivy.properties import ListProperty, StringProperty
from kivy.uix.behaviors import DragBehavior
from kivy.uix.floatlayout import FloatLayout
from kivy.uix.image import Image
from kivy.uix.popup import Popup
from kivy.uix.widget import Widget
from kivy.utils import platform
from kivy.utils import rgba
from tabulate import tabulate

from logic.board import Board
from logic.gates import *
from logic.truth_table import *


Window.maximize()


class ExitPopup(Popup):
    '''Popup that opens when quit button is pressed.'''
    def close_window(self):
        '''Exits program.'''
        sys.exit()


class TruthPopup(Popup):
    '''Popup for truth table generator.'''
    FONT = StringProperty()
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        if platform == 'linux':
            self.FONT = "FreeMonoBold"
        else:
            self.FONT = "LUCON"

    def open(self, root, *args, **kwargs):
```

```
        '''Overwrite of kivy method, inserts selected components expression
into input field.'''
        super().open(*args, **kwargs)
        selected_gate = root.ids["gateCanvas"].get_selected_gates()
        if len(selected_gate) != 1:
            text = 'Enter Boolean Expression Here'
        else:
            text = selected_gate[0].logic_gate.get_expression()
        self.ids["truth_input"].text = text
        self.generate()

    def generate(self):
        '''Using expression in input field, generates a truth table and
diplays it.'''
        input_expression = self.ids["truth_input"].text
        out = generate_truth_table(input_expression)
        if out == "Invalid Input":
            if input_expression == "Enter Boolean Expression Here":
                self.ids["truth_label"].text = "Truth table will appear here"
            else:
                self.ids["truth_label"].text = "Invalid Input."
        else:
            out2 = tabulate(out[0], headers=out[0].keys(), tablefmt="pretty")
            self.ids["truth_label"].text = f"{out[1]}\n{out2}"


class ConnectionLine(Widget):
    '''Class for lines that connects components together'''
    color_state = ListProperty()
    def __init__(self, out_gate, in_gate, in_node, **kwargs):
        super().__init__(**kwargs)
        self.color_states = {None:(1,1,1,1), 0:(1,0,0,1), 1:(0,1,0,1)}
        self.color_state = (1,1,1,1)
        self.out_gate = out_gate
        self.in_gate = in_gate
        self.in_node = in_node
        self.points = []
        self.state = None
        self.inline = Line(points=self.points, width=1.5, cap='round',
joint='round')
        self.outline = Line(points=self.points, width=4, cap='round',
joint='round')
        self.canvas.add(self.inline)
        self.canvas.before.add(Color(rgba=(0,0,0,1)))
        self.canvas.before.add(self.outline)
        self.update_pos()
        self.update_state()

    def get_gates(self):
```

```
        '''Returns tuple: (component line is exiting, component line is
entering)'''
        return (self.out_gate, self.in_gate)

    def get_nodes(self):
        '''Returns tuple: (component line is exiting, node its exiting),
(component line is entering, node its entering))'''
        return ((self.out_gate, -1), (self.in_gate, self.in_node))

    def get_turn_points(self, out_node_pos, in_node_pos):
        '''Returns the extra points to make the line the shape it is <<<<<'''
        x1 = out_node_pos[0]
        x2 = in_node_pos[0]
        y1 = out_node_pos[1]
        y2 = in_node_pos[1]
        x_mid = (x1 + x2)/2
        turn1 = [x_mid, y1]
        turn2 = [x_mid, y2]
        return turn1 + turn2

    def update_state(self):
        '''Updates colour of line to match the output of the component its
exiting.'''
        self.state = self.out_gate.get_state()
        self.color_state = self.color_states[self.state]

    def update_pos(self):
        '''Updates the points of the line to match the position of the
components its connect to.'''
        out_node_pos = self.out_gate.get_node_pos(-1)
        in_node_pos = self.in_gate.get_node_pos(self.in_node)
        self.points = out_node_pos + self.get_turn_points(out_node_pos,
in_node_pos) + in_node_pos
        self.outline.points = self.points
        self.inline.points = self.points


class GateCanvas(FloatLayout):
    '''Canvas where components and connection lines are placed onto'''
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.connection_lines = []
        self.in_connection = None
        self.out_connection = None
        self.gates = []

        self.tool = "move"
        self.board = Board()
        self.gate_dict = {
```

```
            "and":DragAndGate,
            "or":DragOrGate,
            "not":DragNotGate,
            "xor":DragXorGate,
            "switch":DragSwitch,
            "output":DragOutput,
            "clock":DragClock
        }

    def add_connection_line(self, out_gate, in_gate, in_node):
        '''Creates connection line.'''
        line = ConnectionLine(out_gate, in_gate, in_node)
        self.connection_lines.append(line)
        self.add_widget(line)

    def update_connection_lines(self):
        '''Updates the postions of all connection lines on the canvas'''
        for line in self.connection_lines[:]:
            line.update_pos()

    def update_states(self):
        '''Updates the states of all components and connection lines on the
canvas'''
        for gate in self.gates:
            gate.update_state()
        for line in self.connection_lines:
            line.update_state()

    def set_tool(self, tool):
        '''Sets the tool, can be move, connect, disconnect'''
        self.tool = tool
        if tool is "move":
            for gate in self.gates[:]:
                gate.hide_nodes()
        else:
            for gate in self.gates[:]:
                gate.show_nodes()
                self.deselect_gates()

    def on_touch_down(self, touch):
        '''Checks if there is a touch down within the canvas. If touch is
double click, set tool to move. Calls a method based on the current tool if
single click.'''
        if self.collide_point(*touch.pos):
            if touch.is_double_tap and self.tool != "move":
                self.root.set_tool("move")
                return True
            if self.tool == "connect":
```

```
                return self.connect_down(touch)

            if self.tool == "disconnect":
                return self.disconnect_down(touch)

            if self.tool == "move":
                return self.move_down(touch)

    def on_touch_up(self, touch):
        '''Checks if there is a touch up within the canvas, Calls a method
based on the cuurent tool if there is'''
        if self.collide_point(*touch.pos):
            if self.tool == "connect":
                return self.connect_up(touch)
            if self.tool == "disconnect":
                pass
            if self.tool == "move":
                pass
        return super().on_touch_up(touch)

    def connect_down(self, touch):
        '''Checks for node collision on touch down. If collision replace
either out_connection or in_connection attribute with (node, gate) tuple'''
        self.deselect_gates()
        for gate in self.gates[:]:
            if node := gate.get_node_collide(touch):
                if node == -1:
                    self.out_connection = (node, gate)
                else:
                    self.in_connection = (node, gate)
                gate.select_node(node)
                return True
        return True

    def connect_up(self, touch):
        '''Checks for node collision on touch up. If collision, replace either
out_connection or in_connection attribute with (node, gate) tuple.
        If out_connection and in_connection both have values, connects the
component nodes together'''
        for gate in self.gates[:]:
            if node := gate.get_node_collide(touch):
                if node == -1:
                    self.out_connection = (node, gate)
                else:
                    self.in_connection = (node, gate)
                gate.select_node(node)

                if self.in_connection is None:
```

```
                    print("Can't connect output to output")
                elif self.out_connection is None:
                    print("Cant connect input to input")
                else:
                    in_gate = self.in_connection[1]
                    out_gate = self.out_connection[1]
                    in_node = self.in_connection[0]

                    if self.board.connect_gate(in_gate.get_logic_gate(),
out_gate.get_logic_gate(), node=in_node):
                        self.add_connection_line(out_gate, in_gate, in_node)

                    self.update_states()

        for gate in self.gates[:]:
            gate.deselect_nodes()

        self.out_connection = None
        self.in_connection = None
        return True

    def move_down(self, touch):
        '''Calls on_touch_down method for children. If canvas clicked instead
of a child, deselects all gates.'''
        for child in self.gates[::-1]:
            if child.dispatch('on_touch_down', touch):
                return True
        if not self.collide_point(*touch.pos):
            return False
        self.deselect_gates()
        return True

    def disconnect_down(self, touch):
        '''Checks for node collision on touch down. If collision, disconnect
clicked node from any nodes its connected to and remove connection lines.'''
        self.deselect_gates()
        for gate in self.gates[:]:
            if node := gate.get_node_collide(touch):

                if node == -1:
                    for line in self.connection_lines:
                        gate_node_tuple = line.get_nodes()
                        if gate_node_tuple[0][0] == gate:
                            gate2 = gate_node_tuple[1][0]
                            self.board.disconnect_gate(gate2.get_logic_gate(),
gate.get_logic_gate())
                            self.connection_lines.remove(line)
                            self.remove_widget(line)
```

```
                elif node == 1 or node == 2:
                    for line in self.connection_lines:
                        gate_node_tuple = line.get_nodes()
                        if gate_node_tuple[1] == (gate,node):
                            gate2 = gate_node_tuple[0][0]
                            self.board.disconnect_gate(gate.get_logic_gate(),
gate2.get_logic_gate())
                            self.connection_lines.remove(line)
                            self.remove_widget(line)
        self.update_states()
        return True

    def get_selected_gates(self):
        '''Returns a list of all components that are selected.'''
        return [child for child in self.gates[:] if child.is_selected()]

    def deselect_gates(self):
        '''Deselects all selected components.'''
        [child.deselect() for child in self.gates[:]]

    def add_gate(self, gate_type):
        '''Creates new visual component and adds it to canvas, adds logic
component to logic board.'''
        new_gate = self.gate_dict[gate_type](parent_rect=(self.x, self.y,
self.width, self.height))
        self.add_widget(new_gate)
        self.gates.append(new_gate)
        new_gate.root = self.root
        self.board.add_gate(new_gate.get_logic_gate())
        new_gate.update_state()

    def delete_gate(self):
        '''Deletes selected components from canvas and logic board, also
removes any connection lines they are connected to.'''
        if not self.tool == "move":
            self.root.set_tool("move")
        else:
            for gate in self.get_selected_gates():

                #This is probably a terrible way to do this, so find a better
one
                for line in self.connection_lines[:]:
                    if gate in line.get_gates():
                        self.connection_lines.remove(line)
                        self.remove_widget(line)
                        del line
```

```
                self.board.remove_gate(gate.get_logic_gate())
                self.remove_widget(gate)
                self.gates.remove(gate)
                del gate
                self.update_states()

    def clear_canvas(self):
        '''Deletes all components from canvas and logic board'''
        self.board.clear_board()
        self.clear_widgets()
        self.gates = []


class DragGate(DragBehavior, FloatLayout):
    '''Visual components parent class. DOCSRINGS NOT DONE'''
    def __init__(self, parent_rect, **kwargs):
        super().__init__(**kwargs)
        '''Sets kivy properties'''
        self.drag_rectangle = parent_rect
        self.drag_timeout = 500
        self.drag_distance = 5
        self.allow_stretch = True
        self.size_hint = None, None
        self.size = (100,100)
        self.pos = (parent_rect[0]+parent_rect[2])/2,
(parent_rect[1]+parent_rect[3])/2
        '''Sets image of component'''
        self.img = Image(pos = self.pos, size_hint = (1,1))
        self.add_widget(self.img)
        '''Initialises the nodes of the component and adds them to node
array'''
        self.nodes = []
        self.nodes_init()
        '''Creates the selection border and adds to component.'''
        self.border = Line(rounded_rectangle = (self.x, self.y, self.width,
self.height, 10))
        self.canvas.add(Color(0,0,0,1))
        self.canvas.add(self.border)
        '''Creates a parallel logic version of the component. None in parent
class.'''
        self.logic_gate = None
        self.state = None
        '''Creates dragged attribute and selects the component.'''
        self.dragged = False
        self.select()

    def nodes_init(self):
        '''Initialises the nodes of the component, positions them and adds
them to the component widget.'''
```

```
        node_source="GateIcons/node.png"
        self.in_node_1 = Image(source=node_source, size_hint=(None, None),
size=(20,20))
        self.in_node_2 = Image(source=node_source, size_hint=(None, None),
size=(20,20))
        self.out_node = Image(source=node_source, size_hint=(None, None),
size=(20,20))
        self.nodes.append(self.in_node_1)
        self.nodes.append(self.in_node_2)
        self.nodes.append(self.out_node)
        self.add_widget(self.in_node_1)
        self.add_widget(self.in_node_2)
        self.add_widget(self.out_node)
        self.hide_nodes()

    def get_node_pos(self, node):
        '''Return the coordinates of the given node.'''
        return self.get_node(node).center

    def update_nodes(self):
        '''Updates the positions of the components nodes.'''
        self.in_node_1.center=(self.x, self.center_y+26)
        self.in_node_2.center=(self.x, self.center_y-26)
        self.out_node.center=(self.right, self.center_y+2)

    def get_node_collide(self, touch):
        '''Returns node index if the user has clicked a node.'''
        for node in self.nodes:
            if node.collide_point(*touch.pos):
                if node == self.in_node_1:
                    return 1
                elif node == self.in_node_2:
                    return 2
                elif node == self.out_node:
                    return -1
        return False

    def select_node(self, node_index):
        '''Selects the given node.'''
        node = self.get_node(node_index)
        node.color = (0.3, 0.7, 0.7, 1)

    def deselect_node(self, node_index):
        '''Deselects the given node.'''
        node = self.get_node(node_index)
        node.color = (1, 1, 1, 1)

    def deselect_nodes(self):
```

```
        '''Deselects all node of component.'''
        for node in self.nodes:
            node.color = (1, 1, 1, 1)

    def get_node(self, index):
        '''Returns the node object for given node index.'''
        if index == 1:
            return self.in_node_1
        if index == 2:
            return self.in_node_2
        if index == -1:
            return self.out_node

    def on_pos(self, *args, **kwargs):
        '''Listens for change of kivy pos property. Updates position of image,
nodes and selection border. Also tells board to update position of connection
lines.'''
        try:
            self.border.rounded_rectangle = (self.x, self.y, self.width,
self.height, 10)
            self.img.pos = self.pos
            self.update_nodes()
            self.parent.update_connection_lines()
        except AttributeError as e:
            print("Gate not finished initalising", e)

    def is_selected(self):
        '''Returns True if component is currently selected.'''
        return self.selected

    def show_nodes(self):
        '''Shows the components nodes.'''
        for node in self.nodes:
            node.opacity = 1
            node.disabled = False
        self.update_nodes()

    def hide_nodes(self):
        '''Hides the components nodes.'''
        for node in self.nodes:
            node.opacity = 0
            node.disabled = True

    def select(self):
        '''Selects the component.'''
        self.selected = True
        self.border.width = 2
```

```
    def deselect(self):
        '''Deselects the component'''
        self.selected = False
        self.border.width = 0.0001

    def on_touch_down(self, touch):
        '''Checks if user has clicked node, sets dragged attribute to
false.'''
        if self.collide_point(*touch.pos):
            self.dragged = False
        return super().on_touch_down(touch)

    def on_touch_move(self, touch):
        '''Checks if the user drags the component when clicked. Sets dragged
attribute to true, selectes component updates position if user drags within
the drag_timeout time and a distance greater than the drag_distance
distance.'''
        if touch.grab_current is not self:
            return super().on_touch_move(touch)
        self.select()
        self.dragged = True
        if self.x < self.parent.x+5:
            self.x = self.parent.x+5
        elif self.y < self.parent.y+5:
            self.y = self.parent.y+5
        elif self.right > self.parent.right-5:
            self.right = self.parent.right-5
        elif self.top > self.parent.top-5:
            self.top = self.parent.top-5
        else:
            return super().on_touch_move(touch)

    def on_touch_up(self, touch):
        '''When user releases click, checks dragged attribute, if true
component is deselected, if false and the component is already selected the
component is also deselected, if both false component is selected.'''
        if not self.collide_point(*touch.pos):
            return super().on_touch_up(touch)
        if self.dragged:
            self.deselect()
        elif self.selected:
            self.deselect()
        else:
            self.select()
        return super().on_touch_up(touch)

    def get_logic_gate(self):
```

```
        '''Returns the logic component object of this visual component
object.'''
        return self.logic_gate

    def update_state(self):
        '''Updates the state of component to match state of logic
component.'''
        x = self.logic_gate.get_output()
        self.state = x

    def get_state(self):
        '''Returns the components state.'''
        return self.state

class DragSwitch(DragGate):
    '''Switch visual component. User sets state of component'''
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.logic_gate = Switch()
        self.states = {1:"GateIcons/switch_on.png",
0:"GateIcons/switch_off.png"}
        self.img.source = self.states[self.logic_gate.get_output()]

    def nodes_init(self):
        '''Initialises the nodes of the component, positions them and adds
them to the component widget.'''
        node_source="GateIcons/node.png"
        self.out_node = Image(source=node_source, size_hint=(None, None),
size=(20,20))
        self.add_widget(self.out_node)
        self.nodes.append(self.out_node)
        self.hide_nodes()

    def update_nodes(self):
        '''Updates the positions of the components nodes.'''
        self.out_node.center=(self.right, self.y+(self.height//2))

    def get_node_collide(self, touch):
        '''Returns node index if the user has clicked a node.'''
        for node in self.nodes:
            if node.collide_point(*touch.pos):
                if node == self.out_node:
                    return -1
        return False

    def on_touch_up(self, touch):
```

```
        '''When user releases click, checks dragged attribute, if false and
the touch is within the set area, the switch is flipped and dragged is set as
true, then super of method is called.'''
        if not self.dragged:
            if (touch.pos[0] < self.right - 30) and (touch.pos[0] > self.x +
10) and (touch.pos[1] < self.top - 20) and (touch.pos[1] > self.y + 20):
                self.logic_gate.flip()
                self.update_state()
                self.parent.update_states()
                self.dragged = True
        return super().on_touch_up(touch)

    def update_state(self):
        '''Updates the state of component to match state of logic
component.'''
        x = self.logic_gate.get_output()
        self.state = x
        self.img.source = self.states[self.state]

class DragClock(DragGate):
    '''Clock visual component. Kivy scheduler flips state after an interval.
(Uses switch logic component.'''
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.logic_gate = Switch()

        self.states = {1:"GateIcons/clock_on.png",
0:"GateIcons/clock_off.png"}
        self.img.source = self.states[self.logic_gate.get_output()]
        Clock.schedule_interval(self.clock_flip, 1)

    def clock_flip(self, *args):
        '''Flips state of logic component, if AttributeError, the clock has
been deleted so it stop the kivy schedule.'''
        self.logic_gate.flip()
        self.update_state()
        try:
            self.parent.update_states()
        except AttributeError as e:
            print("Clock must have been deleted")
            Clock.unschedule(self.clock_flip)

    def nodes_init(self):
        '''Initialises the nodes of the component, positions them and adds
them to the component widget.'''
        node_source="GateIcons/node.png"
        self.out_node = Image(source=node_source, size_hint=(None, None),
size=(20,20))
```

```
        self.add_widget(self.out_node)
        self.nodes.append(self.out_node)
        self.hide_nodes()

    def update_nodes(self):
        '''Updates the positions of the components nodes.'''
        self.out_node.center=(self.right, self.y+(self.height//2))

    def get_node_collide(self, touch):
        '''Returns node index if the user has clicked a node.'''
        for node in self.nodes:
            if node.collide_point(*touch.pos):
                if node == self.out_node:
                    return -1
        return False

    def update_state(self):
        '''Updates the state of component to match state of logic
component.'''
        x = self.logic_gate.get_output()
        self.state = x
        self.img.source = self.states[self.state]

class DragOutput(DragGate):
    '''Output visual component. Displays its state.'''
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.logic_gate = Output()
        self.states = {1:"GateIcons/output_on.png",
0:"GateIcons/output_off.png", None:"GateIcons/output_empty.png"}
        self.update_state()

    def nodes_init(self):
        '''Initialises the nodes of the component, positions them and adds
them to the component widget.'''
        node_source="GateIcons/node.png"
        self.in_node_1 = Image(source=node_source, size_hint=(None, None),
size=(20,20))
        self.add_widget(self.in_node_1)
        self.nodes.append(self.in_node_1)
        self.hide_nodes()

    def update_nodes(self):
        '''Updates the positions of the components nodes.'''
        self.in_node_1.center=(self.x, self.center_y)

    def get_node_collide(self, touch):
        '''Returns node index if the user has clicked a node.'''
```

```
        for node in self.nodes:
            if node.collide_point(*touch.pos):
                if node == self.in_node_1:
                    return 1
        return False

    def update_state(self):
        '''Updates the state of component to match state of logic
component.'''
        x = self.logic_gate.get_output()
        self.state = x
        self.img.source = self.states[self.state]


class DragAndGate(DragGate):
    '''And Gate visual component.'''
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.img.source = "GateIcons/and.png"
        self.logic_gate = And_Gate()

class DragOrGate(DragGate):
    '''And Or visual component.'''
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.img.source = "GateIcons/or.png"
        self.logic_gate = Or_Gate()

class DragXorGate(DragGate):
    '''And Xor visual component.'''
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.img.source = "GateIcons/xor.png"
        self.logic_gate = Xor_Gate()

class DragNotGate(DragGate):
    '''Not Gate visual component.'''
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.img.source = "GateIcons/not.png"
        self.logic_gate = Not_Gate()

    def nodes_init(self):
        '''Initialises the nodes of the component, positions them and adds
them to the component widget.'''
        node_source="GateIcons/node.png"
        self.in_node_1 = Image(source=node_source, size_hint=(None, None),
size=(20,20))
```

```
        self.out_node = Image(source=node_source, size_hint=(None, None),
size=(20,20))
        self.add_widget(self.in_node_1)
        self.add_widget(self.out_node)
        self.nodes.append(self.in_node_1)
        self.nodes.append(self.out_node)
        self.hide_nodes()

    def update_nodes(self):
        '''Updates the positions of the components nodes.'''
        self.in_node_1.center=(self.x, self.center_y+2)
        self.out_node.center=(self.right, self.center_y+2)

    def get_node_collide(self, touch):
        '''Returns node index if the user has clicked a node.'''
        for node in self.nodes:
            if node.collide_point(*touch.pos):
                if node == self.in_node_1:
                    return 1
                elif node == self.out_node:
                    return -1
        return False


class MainWindow(Widget):
    '''Main Window class'''
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.ids["gateCanvas"].root = self
        self.toggles = ["connectToggle", "moveToggle", "disconnectToggle"]

    def set_tool(self, tool):
        '''Calls Gate Canvases set_tool method.'''
        self.ids["gateCanvas"].set_tool(tool)
        toggle = f"{tool}Toggle"
        if self.ids[toggle].state == 'down':
            pass
        else:
            for tog in self.toggles:
                if tog == toggle:
                    self.ids[tog].state = 'down'
                else:
                    self.ids[tog].state = 'normal'

    def clear_canvas(self):
        '''Calls Gate Canvases clear_canvas method.'''
        self.ids["gateCanvas"].clear_canvas()
        self.set_tool("move")
```

```
    def add_gate(self, gate_type):
        '''Calls Gate Canvases add_gate method.'''
        self.ids["gateCanvas"].add_gate(gate_type)

    def delete_gate(self):
        '''Calls Gate Canvases delete_gate method.'''
        self.ids["gateCanvas"].delete_gate()
        self.set_tool("move")


kv = Builder.load_file("LogicSim.kv")
'''Loads the kivy file'''

class LogicGateSimulator(App):
    '''Kivy App class.'''

    RED = StringProperty('FF0000')
    DARK_GREY = StringProperty('222222')
    BLUE = StringProperty('748bb4')
    LIGHT_GREY = StringProperty('DCDCDC')
    ORANGE = StringProperty('f1ae80')
    WHITE = StringProperty('FFFFFF')
    BLACK = StringProperty('000000')

    def build(self):
        '''Builds the kivy app.'''
        self.icon = "GateIcons/and.png"
        return MainWindow()

if __name__ == '__main__':
    LogicGateSimulator().run()
```

GUI (kvlang)

```
#:kivy 2.0
#:import Factory kivy.factory.Factory

<GridLayout>:
    canvas.before:
        Color:
            rgba: rgba(app.DARK_GREY)
        Rectangle:
            pos: self.pos
            size: self.size

<MyGridLayout@GridLayout>:
    canvas.before:
        Color:
            rgba: rgba(app.BLUE)
        Rectangle:
            pos: self.pos
            size: self.size

<ConnectionLine>:
    canvas:
        Color:
            rgba: self.color_state

<ActionButton>:
    font_size:15
    color: rgba(app.WHITE)

<GateButton@Button>:
    used: False
    size_hint: None, None
    background_normal: ''
    background_color: rgba(app.LIGHT_GREY)
    canvas:
        Color:
            rgba: rgba(app.RED) if not self.used else rgba(app.DARK_GREY)
        Line:
            width: 2
            rectangle: self.x, self.y, self.width, self.height

<ToolToggleButton@ToggleButton>:
    font_size: 15
    color: rgba(app.BLACK)
    size_hint: None, None
    allow_no_selection: False
    background_normal: ''
```

```
    background_color: rgba(app.LIGHT_GREY)
    canvas:
        Color:
            rgba: rgba(app.DARK_GREY)
        Line:
            width: 2
            rectangle: self.x, self.y, self.width, self.height

<ToolButton@Button>:
    font_size: 15
    color: rgba(app.BLACK)
    size_hint: None, None
    background_normal: ''
    background_color: rgba(app.LIGHT_GREY)
    canvas:
        Color:
            rgba: rgba(app.DARK_GREY)
        Line:
            width: 2
            rectangle: self.x, self.y, self.width, self.height

<GateLabel@Label>:
    font_size: 25
    color: rgba(app.BLACK)

<ExitPopup>:
    auto_dismiss: True
    title_color: rgba(app.WHITE)
    title_size: 18
    title: "Are you sure you want to quit?\nAny unsaved progress will be lost"
    size_hint: None, None
    size: 350,150
    BoxLayout:
        spacing: 10
        Button:
            color: rgba(app.BLACK)
            text: 'Cancel'
            size_hint: 0.5, None
            height: 50
            background_normal: ''
            background_color: rgba(app.LIGHT_GREY)
            on_release: root.dismiss()
        Button:
            color: rgba(app.BLACK)
            color:
            text: 'Quit'
            size_hint: 0.5, None
            height: 50
```

```
            background_normal: ''
            background_color: rgba(app.LIGHT_GREY)
            on_release: root.close_window()

<TruthPopup>:
    auto_dismiss: True
    title_color: rgba(app.WHITE)
    title_size: 18
    title: "Truth Table Generator"
    size_hint: 0.6, 0.8
    BoxLayout:
        orientation: "vertical"
        padding: 10
        ScrollView:
            size_hint: 1, 1
            do_scroll_x: False
            effect_cls: "ScrollEffect"
            Label:
                id: truth_label
                size_hint_y: None
                text_size: self.width, None
                height: self.texture_size[1]
                halign: "center"
                bold: True
                font_name: root.FONT
                font_size: 20
                color: rgba(app.WHITE)
        GridLayout:
            size_hint: 1, None
            height: 60
            rows: 1
            columns: 3
            padding: 5
            spacing: 10
            TextInput:
                id: truth_input
                text: "Enter Boolean Expression"
                font_size: 30
                size_hint: 1, 1
                multiline: False
                on_text_validate: root.generate()
            Button:
                color: rgba(app.BLACK)
                text: 'Cancel'
                on_release: root.dismiss()
                size_hint: None, 1
                width: 100
                background_normal: ''
```

```
            background_color: rgba(app.LIGHT_GREY)
        Button:
            color: rgba(app.BLACK)
            text: 'Generate'
            on_release: root.generate()
            size_hint: None, 1
            width: 100
            background_normal: ''
            background_color: rgba(app.LIGHT_GREY)

<MainWindow>:

    GridLayout:
        size: root.width, root.height
        rows:2
        orientation: "tb-lr"
        ActionBar:
            height: 40
            pos_hint: {"top":1}
            ActionView:
                padding: [0,0,10,0]
                ActionPrevious:
                    with_previous: False
                    font_size: 50
                    markup: True
                    title: "[b]Logic Gate Simulator[/b]"
                    app_icon: ""
                    app_icon_width: 0
                ActionButton:
                    text: "Truth Table"
                    on_release: Factory.TruthPopup().open(root)
                ActionButton:
                    text: "Save"
                ActionButton:
                    text: "Load"
                ActionButton:
                    text: "Help"
                ActionButton:
                    text: "Quit"
                    on_release: Factory.ExitPopup().open()

        GridLayout:
            size: self.parent.width, self.parent.height
            orientation: "tb-lr"
            cols: 3
            padding: [10,0,10,10]
            spacing: 10
            ScrollView:
```

```
size_hint: None, 1
width: 200
effect_cls: "ScrollEffect"
canvas.before:
    Color:
        rgba: rgba(app.BLUE)
    Rectangle:
        pos: self.pos
        size: self.size
MyGridLayout:
    size_hint_y: None
    height: 900
    cols: 1
    rows: 2
    padding: 5
    Label:
        size_hint: 1,0.1
        markup: True
        font_size: 40
        color: rgba(app.BLACK)
        text: "[b]Gates[/b]"
    MyGridLayout:
        cols: 2
        size: self.parent.size
        GateLabel:
            text: "And"
        GateButton:
            id: And
            on_press:
                root.add_gate("and")
                self.used = True

            Image:
                source: 'GateIcons/and.png'
                width: self.parent.width * 0.9
                height: self.parent.height * 0.9
                center_x: self.parent.center_x
                center_y: self.parent.center_y

        GateLabel:
            text: "Or"
        GateButton:
            id: Or
            on_press:
                root.add_gate("or")
                self.used = True

            Image:
```

```
            source: 'GateIcons/or.png'
            width: self.parent.width * 0.9
            height: self.parent.height * 0.9
            center_x: self.parent.center_x
            center_y: self.parent.center_y

    GateLabel:
        text: "Not"
    GateButton:
        id: Not
        on_press:
            root.add_gate("not")
            self.used = True

        Image:
            source: 'GateIcons/not.png'
            width: self.parent.width * 0.9
            height: self.parent.height * 0.9
            center_x: self.parent.center_x
            center_y: self.parent.center_y

    GateLabel:
        text: "Xor"
    GateButton:
        id: Xor
        on_press:
            root.add_gate("xor")
            self.used = True

        Image:
            source: 'GateIcons/xor.png'
            width: self.parent.width * 0.9
            height: self.parent.height * 0.9
            center_x: self.parent.center_x
            center_y: self.parent.center_y

    GateLabel:
        text: "Switch"
    GateButton:
        id: Input
        on_press:
            root.add_gate("switch")
            self.used = True

        Image:
            source: 'GateIcons/switch_menu.png'
            width: self.parent.width * 0.9
            height: self.parent.height * 0.9
```

```
                        center_x: self.parent.center_x
                        center_y: self.parent.center_y

                GateLabel:
                    text: "Output"
                GateButton:
                    id: Output
                    on_press:
                        root.add_gate("output")
                        self.used = True

                    Image:
                        source: 'GateIcons/output_menu.png'
                        width: self.parent.width * 0.9
                        height: self.parent.height * 0.9
                        center_x: self.parent.center_x
                        center_y: self.parent.center_y

                GateLabel:
                    text: "Clock"
                GateButton:
                    id: Clock
                    on_press:
                        root.add_gate("clock")
                        self.used = True

                    Image:
                        source: 'GateIcons/clock_menu.png'
                        width: self.parent.width * 0.9
                        height: self.parent.height * 0.9
                        center_x: self.parent.center_x
                        center_y: self.parent.center_y

GateCanvas:
    id: gateCanvas
    canvas.before:
        Color:
            rgba: rgba(app.LIGHT_GREY)
        Rectangle:
            pos: self.pos
            size: self.size
ScrollView:
    size_hint: None, 1
    width: 110
    effect_cls: "ScrollEffect"
    canvas.before:
        Color:
            rgba: rgba(app.ORANGE)
```

```
            Rectangle:
                pos: self.pos
                size: self.size
        GridLayout:
            canvas.before:
                Color:
                    rgba: rgba(app.ORANGE)
                Rectangle:
                    pos: self.pos
                    size: self.size
            orientation: "tb-lr"
            size_hint_y: None
            width: 110
            height: 700
            cols: 1
            padding: 5
            spacing: 10
            Label:
                color: rgba(app.BLACK)
                markup: True
                font_size: 30
                size_hint_y: None
                text: "[b]Tools[/b]"
            ToolToggleButton:
                id: connectToggle
                group: "tools"
                text: "Connect"
                on_press: root.set_tool("connect")
            ToolToggleButton:
                id: disconnectToggle
                group: "tools"
                text: "Disconnect"
                on_press: root.set_tool("disconnect")
            ToolToggleButton:
                id: moveToggle
                state: "down"
                group: "tools"
                text: "Move"
                on_press: root.set_tool("move")
            ToolButton:
                text: "Delete"
                on_press: root.delete_gate()
            ToolButton:
                text: "Clear"
                on_press: root.clear_canvas()
```

Testing

Overview

The testing tables below will refer to the specific objectives in the analysis section. The testing will be split into two sections.

Section 1 will use python unit tests to test that the truth table, logic components and board work correctly and achieve their objectives. The testing is automated using python's unit tests module. The tests are split into test case classes and test methods which I will write to set up the test correctly then using an assert method compare the output to the correct output I have given it. The testing code can be seen at the end of this section with references to the classes of the test cases.

Section 2 will be a video test that shows that the GUI works as it should and that it achieves its objectives. The video is timestamped for each test, some tests do not have timestamps as they were not explicitly tested, but they can be seen throughout the video. The video will have repeats of tests in section one but using the GUI and tests which are not directly referencing the objectives. The video is uploaded to YouTube and the link below. The video testing table will refer to the timestamps in the video for each test.

Code Testing Table

Test No.	Description	Objective	Reference	Passed/ Failed	Notes
1	When a component is created it is the correct type and has no inputs or outputs.	1.1, 1.1.3.1	TestCreatingComponents	Passed	
2	A component with no inputs has an output None, unless the component is a switch then output is default to 0.	1.1.5	TestCreatingComponents	Passed	
3	The switch components flip method flips the state of the switch.	1.1.6	TestCreatingComponents	Passed	
4	Each components evaluate method return the correct output.	1.1.5	TestGateEvaluateMethods	Passed	
5	When a board is created it has an empty gates array.	1.2	TestLogicBoard	Passed	
6	Components can be added to the board's gates array.	1.2.1	TestLogicBoard	Passed	
7	Components can be removed from the board.	1.2.3	TestLogicBoard	Passed	
8	The board connects components together.	1.2.2, 1.1.3	TestConnectingComponents	Passed	
9	The board disconnects components.	1.2.2, 1.1.3	TestDisconnectingComponents	Passed	
10	When the board removes a component from it array, it disconnects everything connected to it.	1.2.3	TestRemovingComponentsWithConnections	Passed	

11	Connecting components together updates their expressions.	1.1.7	TestComponentExpressions	Passed	
12	A component with two valid inputs will have the correct state.	1.1.4, 1.1.5	TestStatesCorrectlyChange	Passed	
13	Changing the state of an input will correctly alter the state of the component.	1.1.4, 1.1.5	TestStatesCorrectlyChange	Passed	
14	The truth table generator will produce a truth table in the form of a dictionary for a valid expression.	2.1.1	TestTruthTable	Passed	
15	The truth table generator will return an error for an invalid expression.	2.1.1	TestTruthTable	Failed	Problem caused by gates that have two inputs, but the inputs do not have valid expressions. Expression looks like: '(None) And (None)'. Should just be None
16	The truth table correctly produces input values for each variable it's given.	2.2	TestTruthTable	Passed	
17	The truth table produces an out column that is accurate to the inputs and expression.	2.3, 2.4	TestTruthTable	Passed	

Video Testing Table

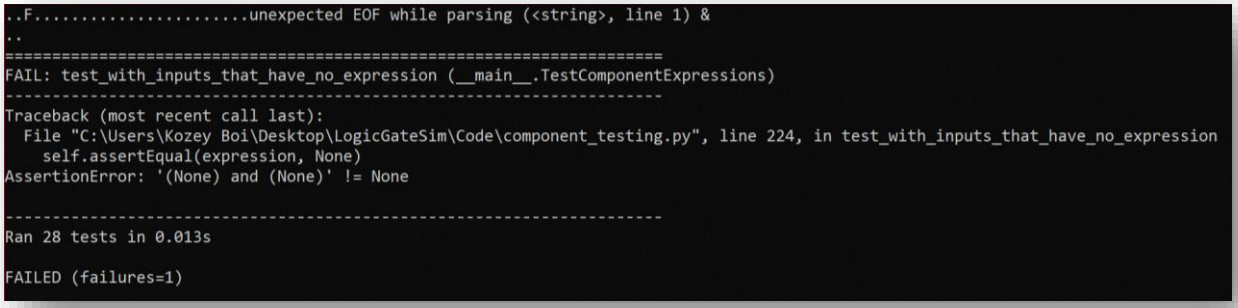
Test No.	Description	Objective	Reference	Passed/ Failed	Notes
18	Window opens to main GUI.		0:00	Passed	
19	Window is maximised on opening.		0:00	Passed	
20	Window is scalable. Side menus stay the same width with the canvas size decreasing.	3.5	0:00	Passed	
21	Buttons in above toolbar is put into drop down menu when window is too small.	3.5	0:20	Passed	
22	If a is window too small to fit full side menus, they allow the user to scroll the menu.	3.5	0:20	Passed	
23	Gate menu consists of all components in objectives.	3.2	0:40	Passed	Menu also contains a clock component which was not an objective.
24	Tool Menu consists of tools: Connect, Disconnect, Move, Delete and Clear.	3.3	0:40	Passed	
25	Move toggle tool is initially selected.		0:40	Passed	

26	Connect, Disconnect and Move are toggle buttons which can only have one active at a time.	3.3.1, 3.3.2, 3.3.3	0:53	Passed	
27	Clicking Delete and clear sets toggle tool to move if it is not already selected.		1:05	Passed	
28	Gate menu buttons start with a red outline to show the component has not been used.	3.2.1	1:18	Passed	
29	Clicking a component button adds that component to the canvas and changes the button outline to black to show it has been used in the current session.	3.2, 3.2.1	1:18	Passed	
30	A component added to the canvas is initially selected.		1:18	Passed	
31	A component that is clicked will be selected or deselected.		1:48	Passed	A Switch will need to be clicked outside of the switch's toggle area.
32	Multiple components can be selected at once.		1:48	Passed	
33	A component can only be selected if the move tool is toggled.		1:48	Passed	Components are still able to be selected when disconnect tool is selected.
34	Clicking and dragging a component selects a component and moves it with the cursor.	3.1.1	2:28	Passed	
35	Clicking the canvas will deselect any selected gates.		-	Passed	
36	Component nodes are only visible if the Connect or Disconnect tool is toggled.		2:38	Passed	
37	With the connect tool select tool selected, dragging from an input node to an output node will connect the nodes.	3.3.1	2:38	Passed	
38	Trying to connect an output node to an output node or input node to an input node will not work.		2:38	Passed	
39	If the connection works a connection line will be added to the canvas connecting the two nodes.	3.1.2	2:38	Passed	
40	The connection line will be coloured to reflect the state of the component it is exiting.	3.1.2.1	-	Passed	White = None Green = True Red = False
41	The connection line will automatically move with the components and adjust its bend to its centre.	3.1.2.2	3:28	Passed	

42	Multiple connections can come from an output node.		3:45	Passed	
43	Only one connection to an input node is allowed.		3:45	Passed	
44	With the disconnect tool selected, clicking a node will disconnect it from any nodes it is connected to.	3.3.2	4:04	Failed	Only disconnects one connection. Should disconnect all.
45	When components are disconnected, connection lines between them are removed.		4:04	Passed	
46	Selecting the connect or disconnect tool will deselect any selected gates.		-	Passed	
47	Using the delete tool will remove any already selected components.	3.34	5:02	Passed	
48	The delete tool will only work if the current toggle tool is move.		5:02	Passed	
49	The clear tool will remove every component from the canvas.	3.3.5	5:26	Passed	
50	Output component displays the state of the component it's connect to.		5:36	Passed	White = None Green = True Red = False
51	Gate components work correctly		5:56	Passed	Tested each gate and some more complicated circuits at the end of video.
52	The truth table button in the toolbar opens a popup for making truth tables.	3.3.6	7:11	Passed	
53	If no component is selected, when the truth table popup is opened there is no truth table, and the input box says enter expression here.	3.3.6.1	7:11	Passed	
54	If a component is selected that has valid inputs, the truth table opens with a truth table for that component already created, the expression in the input box.	3.3.6.2	7:26	Passed	
55	If more than one component is selected the truth table opens in the same state as test 49.	3.3.6.2	7:55	Passed	
56	If a component that has invalid input is selected the truth table opens in the same state as test 49.	3.3.6.2	8:07	Passed	
57	The truth table produces the correct truth table for the given expression.	3.3.6.1	8:20	Passed	

58	The truth table is generated when the generate button is pressed.		8:20	Passed	
59	The truth table popup is closed when the cancel button is pressed.		8:20	Passed	
60	The save button in the toolbar opens the saving menu.	3.4.1	8:55	Failed	
61	The load button in the toolbar opens the loading menu.	3.4.2	8:55	Failed	
62	The help button in the tool bar opens a popup with info on how to use the program.	3.4.3	8:55	Failed	
63	The quit button in the toolbar opens a popup asking, 'Are you sure you want to quit?'.	3.4.4	9:08	Passed	
64	Pressing cancel in the quit popup will close it and return to the program.	3.4.4.1	9:08	Passed	
65	Pressing quit in the quit popup will close the whole program.	3.4.4.1	9:08	Passed	

Code Testing Results



Code Testing Code

```
from logic.gates import *
from logic.board import *
from logic.truth_table import generate_truth_table
import unittest

'''Testing the logic side of the components.'''

class TestCreatingComponents(unittest.TestCase):
    '''Test: When components are made they are of the correct type and have
empty outputs and inputs.'''
    def test_and(self):
        and_gate = And_Gate()
        self.assertIsInstance(and_gate, And_Gate)
        self.assertIsInstance(and_gate, Gate)
        self.assertListEqual(and_gate._input_nodes, [None, None])
        self.assertListEqual(and_gate._output_nodes, [])
        self.assertEqual(and_gate.get_output(), None)

    def test_or(self):
        or_gate = Or_Gate()
        self.assertIsInstance(or_gate, Or_Gate)
        self.assertIsInstance(or_gate, Gate)
        self.assertListEqual(or_gate._input_nodes, [None, None])
        self.assertListEqual(or_gate._output_nodes, [])
        self.assertEqual(or_gate.get_output(), None)

    def test_xor(self):
        xor_gate = Xor_Gate()
        self.assertIsInstance(xor_gate, Xor_Gate)
        self.assertIsInstance(xor_gate, Gate)
        self.assertListEqual(xor_gate._input_nodes, [None, None])
        self.assertListEqual(xor_gate._output_nodes, [])
        self.assertEqual(xor_gate.get_output(), None)
```

```
    def test_not(self):
        not_gate = Not_Gate()
        self.assertIsInstance(not_gate, Not_Gate)
        self.assertIsInstance(not_gate, Gate)
        self.assertListEqual(not_gate._input_nodes, [None, None])
        self.assertListEqual(not_gate._output_nodes, [])
        self.assertEqual(not_gate.get_output(), None)

    def test_switch(self):
        switch = Switch()
        self.assertIsInstance(switch, Switch)
        self.assertListEqual(switch._output_nodes, [])
        self.assertEqual(switch.get_output(), 0)
        switch.flip()
        self.assertEqual(switch.get_output(), 1)

    def test_output(self):
        output = Output()
        self.assertIsInstance(output, Output)
        self.assertListEqual(output._input_nodes, [None])
        self.assertEqual(output.get_output(), None)


class TestGateEvaluateMethods(unittest.TestCase):
    '''Test: The evaluate methods of each gate returns the correct result.'''
    def test_and(self):
        and_gate = And_Gate()
        self.assertEqual(and_gate.evaluate(0,0), 0)
        self.assertEqual(and_gate.evaluate(0,1), 0)
        self.assertEqual(and_gate.evaluate(1,0), 0)
        self.assertEqual(and_gate.evaluate(1,1), 1)

    def test_or(self):
        or_gate = Or_Gate()
        self.assertEqual(or_gate.evaluate(0,0), 0)
        self.assertEqual(or_gate.evaluate(0,1), 1)
        self.assertEqual(or_gate.evaluate(1,0), 1)
        self.assertEqual(or_gate.evaluate(1,1), 1)

    def test_xor(self):
        xor_gate = Xor_Gate()
        self.assertEqual(xor_gate.evaluate(0,0), 0)
        self.assertEqual(xor_gate.evaluate(0,1), 1)
        self.assertEqual(xor_gate.evaluate(1,0), 1)
        self.assertEqual(xor_gate.evaluate(1,1), 0)

    def test_not(self):
```

```
        not_gate = Not_Gate()
        self.assertEqual(not_gate.evaluate(0), 1)
        self.assertEqual(not_gate.evaluate(1), 0)


class TestLogicBoard(unittest.TestCase):
    '''Test: The logic board correctly initiallises, adds, and removes
components from its gates array'''
    def setUp(self):
        self.board = Board()
        self.and_gate = And_Gate()
        self.or_gate = Or_Gate()
        self.xor_gate = Xor_Gate()
        self.not_gate = Not_Gate()
        self.switch1 = Switch()
        self.switch2 = Switch()
        self.output = Output()

    def add_components_to_board(self):
        self.board.add_gate(self.and_gate)
        self.board.add_gate(self.or_gate)
        self.board.add_gate(self.xor_gate)
        self.board.add_gate(self.not_gate)
        self.board.add_gate(self.switch1)
        self.board.add_gate(self.switch2)
        self.board.add_gate(self.output)

    def test_board_initially_empty(self):
        self.assertListEqual(self.board.gates, [])

    def test_adding_components(self):
        self.add_components_to_board()
        self.assertListEqual(self.board.gates, [self.and_gate, self.or_gate,
self.xor_gate, self.not_gate, self.switch1, self.switch2, self.output])

    def test_removing_components(self):
        self.board.add_gate(self.switch1)
        self.board.add_gate(self.switch2)
        self.board.add_gate(self.output)
        self.assertListEqual(self.board.gates, [self.switch1, self.switch2,
self.output])
        self.board.remove_gate(self.switch2)
        self.assertListEqual(self.board.gates, [self.switch1, self.output])

class TestConnectingComponents(unittest.TestCase):
    '''Test: The logic board correctly connects components.'''
    def setUp(self):
        self.board = Board()
```

```
        self.and_gate = And_Gate()
        self.switch1 = Switch()
        self.switch2 = Switch()

    def add_components_to_board(self):
        self.board.add_gate(self.and_gate)
        self.board.add_gate(self.switch1)
        self.board.add_gate(self.switch2)

    def test_connecting_components(self):
        self.add_components_to_board()
        self.board.connect_gate(self.and_gate, self.switch1, 1)
        self.board.connect_gate(self.and_gate, self.switch2, 2)
        self.assertListEqual(self.and_gate._input_nodes, [self.switch1,
self.switch2])
        self.assertIn(self.and_gate, self.switch1._output_nodes)


class TestDisconnectingComponents(unittest.TestCase):
    '''Test: The logic board correctly disconnects components.'''
    def setUp(self):
        self.board = Board()
        self.and_gate = And_Gate()
        self.switch1 = Switch()
        self.switch2 = Switch()

    def add_components_to_board(self):
        self.board.add_gate(self.and_gate)
        self.board.add_gate(self.switch1)
        self.board.add_gate(self.switch2)

    def test_disconnecting_two_input_components(self):
        self.add_components_to_board()
        self.board.connect_gate(self.and_gate, self.switch1, 1)
        self.board.connect_gate(self.and_gate, self.switch2, 2)
        self.board.disconnect_gate(self.and_gate, self.switch1)
        self.assertListEqual(self.and_gate._input_nodes, [None, self.switch2])

class TestRemovingComponentsWithConnections(unittest.TestCase):
    '''Test: The logic board disconnects everything from a component it is
removing.'''
    def setUp(self):
        self.board = Board()
        self.xor_gate = Xor_Gate()
        self.switch1 = Switch()
        self.switch2 = Switch()
        self.output = Output()
```

```
    def add_components_to_board(self):
        self.board.add_gate(self.xor_gate)
        self.board.add_gate(self.switch1)
        self.board.add_gate(self.switch2)
        self.board.add_gate(self.output)

    def test_removing_component_with_connections(self):
        self.add_components_to_board()
        self.board.connect_gate(self.xor_gate, self.switch1, 1)
        self.board.connect_gate(self.xor_gate, self.switch2, 2)
        self.board.connect_gate(self.output, self.xor_gate, 1)

        self.assertListEqual(self.xor_gate._input_nodes, [self.switch1,
self.switch2])
        self.assertListEqual(self.switch1._output_nodes, [self.xor_gate])
        self.assertListEqual(self.switch2._output_nodes, [self.xor_gate])

        self.assertListEqual(self.xor_gate._output_nodes, [self.output])
        self.assertListEqual(self.output._input_nodes, [self.xor_gate])

        self.assertIn(self.xor_gate, self.board.gates)
        self.board.remove_gate(self.xor_gate)
        self.assertListEqual(self.xor_gate._input_nodes, [None, None])
        self.assertListEqual(self.switch1._output_nodes, [])
        self.assertListEqual(self.switch2._output_nodes, [])

        self.assertListEqual(self.xor_gate._output_nodes, [])
        self.assertListEqual(self.output._input_nodes, [None])

class TestComponentExpressions(unittest.TestCase):
    '''Test: The expression update correctly'''
    def setUp(self):
        self.board = Board()
        self.and_gate = And_Gate()
        self.or_gate = Or_Gate()
        self.not_gate = Not_Gate()
        self.switch1 = Switch()
        self.switch2 = Switch()
        self.output = Output()

    def test_expression_none_when_no_input(self):
        expression = self.and_gate.get_expression()
        self.assertEqual(expression, None)

    def test_with_not_enough_inputs(self):
        self.board.connect_gate(self.and_gate, self.switch1, 1)
        expression = self.and_gate.get_expression()
        self.assertEqual(expression, None)
```

```
    def test_with_inputs_that_have_no_expression(self):
        self.board.connect_gate(self.and_gate, self.or_gate, 1)
        self.board.connect_gate(self.and_gate, self.not_gate, 2)
        expression = self.and_gate.get_expression()
        self.assertEqual(expression, None)

    def test_with_valid_inputs(self):
        self.board.connect_gate(self.and_gate, self.switch1, 1)
        self.board.connect_gate(self.and_gate, self.switch2, 2)
        switch_variable1 = self.switch1.get_expression()
        switch_variable2 = self.switch2.get_expression()
        expression = self.and_gate.get_expression()
        valid_expression = f"{switch_variable1} and {switch_variable2}"
        self.assertEqual(expression, valid_expression)

    def test_expression_set_to_none_when_inputs_removed(self):
        self.board.connect_gate(self.and_gate, self.switch1, 1)
        self.board.connect_gate(self.and_gate, self.switch2, 2)
        switch_variable1 = self.switch1.get_expression()
        switch_variable2 = self.switch2.get_expression()
        expression1 = self.and_gate.get_expression()
        valid_expression = f"{switch_variable1} and {switch_variable2}"
        self.assertEqual(expression1, valid_expression)
        self.board.disconnect_gate(self.and_gate, self.switch1)
        self.board.disconnect_gate(self.and_gate, self.switch2)
        expression2 = self.and_gate.get_expression()
        self.assertNotEqual(expression2, valid_expression)
        self.assertEqual(expression2, None)

class TestStatesCorrectlyChange(unittest.TestCase):
    '''Test: The states of components update correctly when an input is
altered.'''
    def setUp(self):
        self.board = Board()
        self.and_gate = And_Gate()
        self.or_gate = Or_Gate()
        self.xor_gate = Xor_Gate()
        self.not_gate = Not_Gate()
        self.switch1 = Switch()
        self.switch2 = Switch()
        self.output = Output()

        self.board.add_gate(self.and_gate)
        self.board.add_gate(self.or_gate)
        self.board.add_gate(self.xor_gate)
        self.board.add_gate(self.not_gate)
        self.board.add_gate(self.switch1)
```

```
        self.board.add_gate(self.switch2)
        self.board.add_gate(self.output)

    def test_and(self):
        self.board.connect_gate(self.and_gate, self.switch1, 1)
        self.board.connect_gate(self.and_gate, self.switch2, 2)
        self.assertEqual(self.and_gate.get_output(), 0) #[0,0] -> 0
        self.switch1.flip()
        self.assertEqual(self.and_gate.get_output(), 0) #[1,0] -> 0
        self.switch1.flip()
        self.switch2.flip()
        self.assertEqual(self.and_gate.get_output(), 0) #[0,1] -> 0
        self.switch1.flip()
        self.assertEqual(self.and_gate.get_output(), 1) #[1,1] -> 1

    def test_or(self):
        self.board.connect_gate(self.or_gate, self.switch1, 1)
        self.board.connect_gate(self.or_gate, self.switch2, 2)
        self.assertEqual(self.or_gate.get_output(), 0) #[0,0] -> 0
        self.switch1.flip()
        self.assertEqual(self.or_gate.get_output(), 1) #[1,0] -> 1
        self.switch1.flip()
        self.switch2.flip()
        self.assertEqual(self.or_gate.get_output(), 1) #[0,1] -> 1
        self.switch1.flip()
        self.assertEqual(self.or_gate.get_output(), 1) #[1,1] -> 1

    def test_xor(self):
        self.board.connect_gate(self.xor_gate, self.switch1, 1)
        self.board.connect_gate(self.xor_gate, self.switch2, 2)
        self.assertEqual(self.xor_gate.get_output(), 0) #[0,0] -> 0
        self.switch1.flip()
        self.assertEqual(self.xor_gate.get_output(), 1) #[1,0] -> 1
        self.switch1.flip()
        self.switch2.flip()
        self.assertEqual(self.xor_gate.get_output(), 1) #[0,1] -> 1
        self.switch1.flip()
        self.assertEqual(self.xor_gate.get_output(), 0) #[1,1] -> 0
    def test_not(self):
        self.board.connect_gate(self.not_gate, self.switch1, 1)
        self.assertEqual(self.not_gate.get_output(), 1) #[0] -> 1
        self.switch1.flip()
        self.assertEqual(self.not_gate.get_output(), 0) #[1] -> 0

    def test_output(self):
        self.board.connect_gate(self.output, self.switch1, 1)
        self.assertEqual(self.output.get_output(), 0) #[0] -> 0
        self.switch1.flip()
```

```
        self.assertEqual(self.output.get_output(), 1) #[1] -> 1

class TestTruthTable(unittest.TestCase):
    '''Test: Correct truth tables are produced for valid expressions'''
    def setUp(self):
        self.board = Board()
        self.and_gate = And_Gate()
        self.switch1 = Switch()
        self.switch2 = Switch()
        self.output = Output()
        self.board.add_gate(self.and_gate)
        self.board.add_gate(self.switch1)
        self.board.add_gate(self.switch2)
        self.board.add_gate(self.output)

    def test_with_valid_expression(self):
        '''generate_truth_table returns a tuple with the truth table
dictionary and the expression string.
        Tests that the truth dictionary is correct as expression will not
change.'''
        expression = "A AND B"
        tt = generate_truth_table(expression)
        self.assertDictEqual(
            tt[0],
            {
                'a':[0,0,1,1],
                'b':[0,1,0,1],
                'OUT':[0,0,0,1]
            }
        )
        self.assertEqual(tt[1], expression)

    def test_with_invalid_expression(self):
        expression = "AND"
        tt = generate_truth_table(expression)
        self.assertEqual(tt, "Invalid Input")


if __name__ == '__main__':
    unittest.main()
```

Video Testing Link

https://youtu.be/HILKC_kly70

Evaluation

Overview

All objectives related to the logic circuits and user interface were completed to some extent, the saving, loading, and help objectives were not completed mostly due to time constraints. Most of the tests passed with no problems and the very few tests that failed are minor problems that do not cause any errors or affect the main objectives of the program. The logic code could be designed much better than currently, the board storing the connections between each component would simplify the component classes and allow for more efficient processing.

Objectives Not Met

The saving and loading of circuits would have been great to include but unfortunately due to the nature of how the GUI and logic interact, the implementation would have taken too long to develop. The GUI is not 100% independent of the logic so saving would need to account for the saving the Kivy elements of the program, of which my understanding is limited.

The help menu also was not implemented, it would have been easy to do as it would just need to show an information page that could have been just a picture, but the creating of that page would have been time consuming to create.

Objectives Partially Met

Test 15 showed that the truth table generator would not correctly handle an invalid expression obtained from an incomplete circuit. This is due to the expression updates in the component not accounting for 'None' expressions at a depth greater than 1 below the selected component. E.g. if a component was given connected to two inputs that have invalid expressions, say an and gate had two not gates as inputs with no inputs for the nots, then the component was connected to an output, the output would see the components has two inputs and would treat it as valid. This creates a problem of giving an expression (none) and (none). This does not happen if the output is connected before the two inputs of the and gate and does not affect the accuracy of the truth table generation as there wouldn't be one for the incomplete circuit anyway.

Test 44 also failed as some components were able to be selected when the disconnect tool was active. Toggling the disconnect tool would deselect all components but on a few occasions a component could be selected if clicked. The component cannot be interacted with as if it was selected, i.e. it cannot be dragged or deleted, and instead is just a visual error. The problem was not full repeatable so I could not deduce the reason behind the error.

Improvements That Can Be Made

The program is suitable for the circuits that a GCSE student would make, however the program would be better if it could accommodate to A-Level students as well. Most of the circuits and expressions needed for the A level course can be made, but any circuit with a cycle is not possible due to the way the logic is evaluated. D type flip flops are an

important circuit for A-level students and is not possible because of this restriction. A new evaluation method would be needed to allow for cycles in circuits.

Testing was not done for cycles although it should have been included in the video as trying to make a circuit with cycles crashes the program due to a recursion depth error. This is a big problem with the program and should be fixed. Currently, as the intended users would not need to make a circuit with cycles, the error has not been removed and instead would advise users not to use cycles.

User Feedback

...